

РАБОЧАЯ ПРОГРАММА ДИСЦИПЛИНЫ

Б1.О.18 БАЗЫ ДАННЫХ

(индекс, наименование дисциплины в соответствии с учебным планом)

09.03.03 Прикладная информатика

(код, наименование направления подготовки/специальности)

Корпоративные информационные системы управления

(наименование образовательной программы)

очная, заочная

(форма обучения)

Год набора – 2025

Нижний Новгород, 2025 г.

Автор(ы)-составитель(и) РПД:

Автор(ы)-составитель(и):

Заведующий кафедрой информатики и
информационных технологий, кандидат технических
наук, доцент

(ученая степень и(или) ученое звание, должность, наименование кафедры)

И.И. Гребенюк
(Ф.И.О.)

Доцент кафедры информатики и информационных
технологий, кандидат технических наук

(ученая степень и(или) ученое звание, должность, наименование кафедры)

А.Б. Гордеев
(Ф.И.О.)

Заведующий кафедрой информатики и
информационных технологий, кандидат технических
наук, доцент

(ученая степень и(или) ученое звание, должность, наименование кафедры)

И.И. Гребенюк
(Ф.И.О.)

РПД Б1.О.18 «Базы данных» одобрена на заседании кафедры
информатики и информационных технологий

Протокол от 23 сентября 2025 г. № 2.

СОДЕРЖАНИЕ

1. Перечень планируемых результатов обучения по дисциплине, соотнесенных с планируемыми результатами освоения образовательной программы
2. Объем и место дисциплины в структуре образовательной программы
3. Содержание и структура дисциплины
4. Типы оценочных материалов, показатели и критерии их оценивания
5. Формы аттестации, типовые оценочные материалы для текущего контроля успеваемости обучающихся, критерии и шкалы оценивания по контрольным точкам
6. Формы промежуточной аттестации, критерии и шкала оценивания, типовые оценочные материалы по дисциплине
7. Методические материалы по освоению дисциплины
8. Учебная литература и ресурсы информационно-телекоммуникационной сети «Интернет»
9. Материально-техническая база, информационные технологии, программное обеспечение и информационные справочные системы

1. Перечень планируемых результатов обучения по дисциплине, соотнесенных с планируемыми результатами освоения образовательной программы

Дисциплина Б1.О.18 «Базы данных» обеспечивает формирование у обучающихся следующих универсальных, общепрофессиональных и профессиональных компетенций*:

ОТФ/ТФ и реквизиты ПС	Код компетенции	Наименование Компетенции	Код индикатора достижения компетенций	Наименование индикатора достижения компетенций	Образовательный результат
	ОПК-2	Способен понимать принципы работы современных информационных технологий и программных средств, в том числе отечественного производства, и использовать их при решении задач профессиональной деятельности.	ОПК-2.2	Способен применять программные средства в процессе решения задачи профессиональной деятельности	ОПК-2.2. 3-1. Знает современные информационные технологии и программные средства, в том числе отечественного производства. ОПК-2.2. У-1. Умеет выбирать современные информационные технологии и программные средства, в том числе отечественного производства при решении задач профессиональной деятельности.

ОТФ/ТФ и реквизиты ПС	Код компетенции	Наименован ие Компетенции	Код индикатора достижения компетенций	Наименование индикатора достижения компетенций	Образовате льный результат
	ОПК ОС-10	Способен решать комплекс задач по созданию, эксплуатации, безопасности и развитию прикладных информацион ных систем	ОПК ОС-10.2	Способен реализовать алгоритмы решения комплекса задач, связанных с обеспечением безопасности в процессе создания, эксплуатации и развития прикладных информационн ых систем	ОПК-10.2. 3- 1. Знает типовые программно- аппаратные средства и системы защиты информации от несанкциони рованного доступа в компьютерн ую среду. ОПК-10.2. У-1. Умеет использоват ь типовые программно- аппаратные средства и системы защиты информации от несанкциони рованного доступа в компьютерн ую среду.
	ОПК ОС-11	Способен совершенство вать информацион ные среды с учетом последних значимых разработок и открытий в области ИТ, новых программных продуктов, направленных	ОПК ОС-11.1	Способен применять новые методы и разработки в области ИТ для оптимизации производствен ных процессов	ОПК-11.1. 3- 1. Знает основные понятия и компоненты банков данных, подходы к построению БД, особенности реляционной модели и их влияние на проектирова ние БД, языки

ОТФ/ТФ и реквизиты ПС	Код компетенции	Наименован ие Компетенции	Код индикатора достижения компетенций	Наименование индикатора достижения компетенций	Образовате льный результат
		на оптимизацию всех видов производстве нных процессов посредством информацион ных технологий и автоматизаци и			описания и манипулиро вания данными, классификац ию и способы задания ограничений целостности, модель “сущность - связь” и ее основные конструктив ные элементы, технологию оперативной обработки транзакций. ОПК-11.1. У-1. Умеет построить модель предметной области, спроектиров ать реляционну ю базу данных (определить состав каждой таблицы, типы полей, ключ для каждой таблицы), определить ограничения целостности, формулиров ать запросы к базе данных, осуществляют

ОТФ/ТФ и реквизиты ПС	Код компетенции	Наименован ие Компетенции	Код индикатора достижения компетенций	Наименование индикатора достижения компетенций	Образова тельный результат
					Б декларативн ую и процедурну ю поддержку целостности данных.

2. Объем и место дисциплины в структуре образовательной программы

Индекс дисциплины Б1.О.18 «Базы данных» является дисциплиной обязательной части образовательной программы.

Общая трудоемкость дисциплины составляет 6 зачетных единиц, 216 академических часов / 162 астрономических часа.

По очной форме обучения: количество академических часов, выделенных на контактную работу с преподавателем составляет 82 часов, из них, лекции - 32 часа, практические занятия - 48 часов, консультация – 2 часа. Самостоятельная работа составляет - 94 часов, контроль – 40 часов.

По заочной форме обучения: количество академических часов, выделенных на контактную работу с преподавателем составляет 28 часов, из них, лекции – 12 часов, практические занятия – 14 часов, консультация – 2 часа. Самостоятельная работа составляет 175 часов, контроль – 13 часов.

Форма промежуточной аттестации в соответствии с учебным планом: *зачет с оценкой, экзамен.*

3. Содержание и структура дисциплины

3.1. Структура дисциплины Очная/заочная форма обучения

№ п/п	Наименование тем и (или) разделов		Объем дисциплины, ак.час											Форма текущего контроля успеваемости, промежуточной аттестации	
		ВСЕГО	Контактная работа обучающихся с преподавателем по видам учебных занятий								Самостоятельная работа				
			Период теоретического обучения						Период промежуточной аттестации (сессия)						
			Занятия лекционного типа		Занятия семинарского типа		ИК	КСР	КЭ	Каттэк	Контроль	СРкр	СРэк		СР
			Л	ВЛ	ЛР	ПЗ									
Очная форма обучения															
Тема 1	Введение в базы данных и SQL	10	2			2						1	5	Опрос	
Тема 2	Создание рабочей среды	10	2			2						1	5	Опрос, контрольные задания с развернутым ответом	
Тема 3	Основные операции с таблицами	14	2			4						1	5	Опрос, контрольные задания с развернутым ответом	
Тема 4	Типы данных СУБД PostgreSQL	16	4			4						1	7	Опрос, контрольные задания с развернутым ответом	

№ п/п	Наименование тем и (или) разделов		Объем дисциплины, ак.час											Форма текущего контроля успеваемости, промежуточной аттестации	
		ВСЕГО	Контактная работа обучающихся с преподавателем по видам учебных занятий								Самостоятельная работа				
			Период теоретического обучения						Период промежуточной аттестации (сессия)						
			Занятия лекционного типа		Занятия семинарского типа		ИК	КСР	КЭ	Каттэк	Контроль	СРкр	СРэк	СР	
			Л	ВЛ	ЛР	ПЗ									
Тема 5	Основы языка определения данных	18	6			4						1		7	Опрос, контрольные задания с развернутым ответом
Тема 6	Запросы	14	2			4						1		7	Опрос, контрольные задания с развернутым ответом
Тема 7	Изменение данных	14	2			4						1		7	Опрос, контрольные задания с развернутым ответом
Тема 8	Индексы	14	2			4						1		7	Опрос, контрольные задания с развернутым ответом
Тема 9	Транзакции	16	2			6						1		7	Опрос, контрольные задания с развернутым ответом
Тема 10	Повышение производительности	20	4			6						1		9	Опрос, контрольные задания с развернутым ответом

№ п/п	Наименование тем и (или) разделов		Объем дисциплины, ак.час											Форма текущего контроля успеваемости, промежуточной аттестации	
		ВСЕГО	Контактная работа обучающихся с преподавателем по видам учебных занятий								Самостоятельная работа				
			Период теоретического обучения						Период промежуточной аттестации (сессия)						
			Занятия лекционного типа		Занятия семинарского типа		ИК	КСР	КЭ	Каттэк	Контроль	СРкр	СРэк		СР
			Л	ВЛ	ЛР	ПЗ									
Тема 11	Объектно-ориентированные базы данных	14	2			4						1		7	Опрос, контрольные задания с развернутым ответом
Тема 12	Использование XML при работе с БД	14	2			4						1		7	Опрос, контрольные задания с развернутым ответом
Консультация		2							2						
Промежуточная аттестация		40								40					Зачет с оценкой, экзамен
Итого (в з.е./акад.часах/астр.часах)		6/216/162	32/24			48/36			2/1,5	40/30		12/9		82/61,5	
Заочная форма обучения															
Тема 1	Введение в базы данных и SQL	16	1			1						1		13	Опрос

№ п/п	Наименование тем и (или) разделов		Объем дисциплины, ак.час											Форма текущего контроля успеваемости, промежуточной аттестации	
		ВСЕГО	Контактная работа обучающихся с преподавателем по видам учебных занятий								Самостоятельная работа				
			Период теоретического обучения					Период промежуточной аттестации (сессия)							
			Занятия лекционного типа		Занятия семинарского типа		ИК	КСР	КЭ	Каттэк	Контроль	СРкр	СРэк		СР
			Л	ВЛ	ЛР	ПЗ									
Тема 2	Создание рабочей среды	16	1			1						1		13	Опрос, контрольные задания с развернутым ответом
Тема 3	Основные операции с таблицами	16	1			1						1		13	Опрос, контрольные задания с развернутым ответом
Тема 4	Типы данных СУБД PostgreSQL	16	1			1						1		13	Опрос, контрольные задания с развернутым ответом
Тема 5	Основы языка определения данных	16	1			1						1		13	Опрос, контрольные задания с развернутым ответом
Тема 6	Запросы	16	1			1						1		13	Опрос, контрольные задания с развернутым ответом
Тема 7	Изменение данных	16	1			1						1		13	Опрос, контрольные задания с развернутым ответом

№ п/п	Наименование тем и (или) разделов		Объем дисциплины, ак.час											Форма текущего контроля успеваемости, промежуточной аттестации	
		ВСЕГО	Контактная работа обучающихся с преподавателем по видам учебных занятий								Самостоятельная работа				
			Период теоретического обучения						Период промежуточной аттестации (сессия)						
			Занятия лекционного типа		Занятия семинарского типа		ИК	КСР	КЭ	Каттэк	Контроль	СРкр	СРэк	СР	
			Л	ВЛ	ЛР	ПЗ									
Тема 8	Индексы	16	1			1						1	13	Опрос, контрольные задания с развернутым ответом	
Тема 9	Транзакции	20	1			2						1	16	Опрос, контрольные задания с развернутым ответом	
Тема 10	Повышение производительности	21	1			2						1	17	Опрос, контрольные задания с развернутым ответом	
Тема 11	Объектно-ориентированные базы данных	16	1			1						1	13	Опрос, контрольные задания с развернутым ответом	
Тема 12	Использование XML при работе с БД	16	1			1						1	13	Опрос, контрольные задания с развернутым ответом	
Консультация		2							2						
Промежуточная аттестация		13								13				Зачет с оценкой, экзамен	

№ п/п	Наименование тем и (или) разделов		Объем дисциплины, ак.час											Форма текущего контроля успеваемости, промежуточной аттестации	
		ВСЕГО	Контактная работа обучающихся с преподавателем по видам учебных занятий								Самостоятельная работа				
			Период теоретического обучения				Период промежуточной аттестации (сессия)								
			Занятия лекционного типа		Занятия семинарского типа		ИК	КСР	КЭ	Каттэк	Контроль	СРкр	СРэк		СР
			Л	ВЛ	ЛР	ПЗ									
Итого (в з.е./акад.часах/астр.часах):		6/216/162	12/9			14/ 10,5			2/1,5	13/9,75		12/9	163/122,25		

Используемые сокращения:

Л – лекции - занятия, предусматривающие преимущественную передачу учебной информации обучающимся педагогическими работниками организации и (или) лицами, привлекаемыми организацией к реализации образовательных программ на иных условиях,).

ВЛ – видео лекции.

ЛР – лабораторные работы.

ПЗ – практические занятия (за исключением лабораторных работ).

ИК – индивидуальные консультации.

КСР – контроль самостоятельной работы

КЭ – консультации перед экзаменом

Каттэк – контактная работа на аттестацию в период экзаменационных сессий

Контроль - контактная работа на аттестацию в период экзаменационных сессий для заочной формы обучения

СРкр – самостоятельная работа на подготовку курсовой работы/ курсового проекта.

СРэк – самостоятельная работа на подготовку к экзамену.

СР – самостоятельная работа в семестре на подготовку к учебным занятиям.

3.2. Содержание дисциплины

Тема 1 Введение в базы данных и SQL ОПК-2.2

Что такое базы данных и зачем они нужны. Основные понятия реляционной модели. Что такое язык SQL. Концептуальный уровень проектирования, Логический уровень проектирования, Физический уровень проектирования. Проблемы концептуального уровня проектирования. Проблемы логического уровня проектирования: выбор ключей, восприятие схем данных. Проблемы физического уровня проектирования: выбор типов данных, масштабируемость, гибкость, обработка логики, скорость работы СУБД.

Тема 2 Создание рабочей среды ОПК-2.2

Установка СУБД. Программа psql — интерактивный терминал PostgreSQL. Установка PostgreSQL в Docker. Запуск контейнера PostgreSQL в Docker. Как подключиться к PostgreSQL в контейнере. Запуск PostgreSQL с помощью docker-compose.

Тема 3 Основные операции с таблицами ОПК-2.2

Таблицы в SQL: типы и операции. Что такое таблицы в SQL. Типы таблиц в SQL: обычные таблицы, секционированные таблицы, временные таблицы SQL, системные таблицы, широкие таблицы, связанные таблицы SQL. Основные операции над таблицами в SQL: создание таблиц, вставка данных, выборка данных, обновление данных, удаление данных.

Тема 4 Типы данных СУБД PostgreSQL ОПК-2.2

Числовые типы: Целочисленные типы; Числа с произвольной точностью; Типы с плавающей точкой; Последовательные типы. Денежные типы: Символьные типы. Двоичные типы данных: Шестнадцатеричный формат bytea; Формат спецпоследовательностей bytea. Типы даты/времени: Ввод даты/времени; Вывод даты/времени; Часовые пояса; Ввод интервалов; Вывод интервалов. Логический тип. Типы перечислений: Объявление перечислений: Порядок; Безопасность типа: Тонкости реализации. Геометрические типы: Точки; Прямые; Отрезки; Прямоугольники; Пути; Многоугольники; Окружности. Типы, описывающие сетевые адреса: inet; cidr; Различия inet и cidr; macaddr; macaddr8. Битовые строки. Типы, предназначенные для текстового поиска: tsvector; tsquery; Тип UUID. Тип XML: Создание XML-значений. Обработка кодировки. Обращение к XML-значениям. Типы JSON: Синтаксис вводимых и выводимых значений JSON. Проектирование документов JSON. Проверки на вхождение и существование jsonb. Индексация jsonb. Обращение по индексу к элементам jsonb. Трансформации. Тип jsonpath. Массивы: Объявления типов массивов; Ввод значения массива; Обращение к массивам; Изменение массивов; Поиск значений в массивах; Синтаксис вводимых и выводимых значений массива; Составные типы: Объявление составных типов;

Конструирование составных значений; Обращение к составным типам; Изменение составных типов; Использование составных типов в запросах; Синтаксис вводимых и выводимых значений составного типа. Диапазонные типы: Встроенные диапазонные и мультидиапазонные типы: Включение и исключение границ; Неограниченные (бесконечные) диапазоны; Ввод/вывод диапазонов; Конструирование диапазонов и мультидиапазонов; Типы дискретных диапазонов; Определение новых диапазонных типов; Индексация; Ограничения для диапазонов. Типы доменов. Идентификаторы объектов. Тип `pg_lsn`. Псевдотипы

Тема 5 Основы языка определения данных ОПК-2.2

Значения по умолчанию и ограничения целостности. Создание и удаление таблиц. Модификация таблиц. Представления. Схемы базы данных.

Тема 6 Запросы ОПК-2.2

Понятие запроса. Классификация запросов по функциональному признаку. Запросы-действия: запросы на создание таблицы, запросы на добавление данных, запросы на изменение данных, запросы на удаление данных. Запросы на выборку: запросы с условием, запросы с групповыми операциями. Понятие скрипта (сценария). Представление, как один из видов запроса. Триггер, как один из видов запроса. Процедура, как один из видов запроса. *Дополнительные возможности команды SELECT*. Соединения. Агрегирование и группировка. Подзапросы.

Тема 7 Изменение данных ОПК-2.2

Вставка строк в таблицы. Обновление строк в таблицах. Удаление строк из таблиц.

Тема 8 Индексы ОПК-2.2

Общая информация. Индексы по нескольким столбцам. Уникальные индексы. Индексы на основе выражений. Частичные индексы

Тема 9 Транзакция. ОПК-2.2, ОПК ОС-10.2

Общая информация. Уровень изоляции Read Uncommitted. Уровень изоляции Read Committed. Уровень изоляции Repeatable Read. Уровень изоляции Serializable. Блокировки

Тема 10 Повышение производительности. ОПК-2.2, ОПК ОС-10.2, ОПК ОС-11.1

Основные понятия. Методы просмотра таблиц. Методы формирования соединений наборов строк. Управление планировщиком. Оптимизация запросов. Терминология и основы системы безопасности SQL Server. Логины (имена входа). Выбор типа логина. Создание логина и

настройка его параметров. Режимы аутентификации. Аудит попыток входа. Понятие серверных ролей. Разрешения на уровне сервера. Пользователи баз данных. Понятие пользовательской схемы. Создание, изменение и удаление пользователей базы данных. Роли баз данных. Предоставление прав на объекты в базе данных. Роли приложений. Изменение контекста выполнения. Применение сертификатов и шифрование данных в Progress SQL.

Тема 11 Объектно-ориентированные базы данных ОПК-2.2, ОПК ОС-10.2, ОПК ОС-11.1

Понятие объектно-ориентированных баз данных (ООБД) и объектно-ориентированных систем управления базами данных (ООСУБД). Характеристики ООБД. Достоинства и недостатки модели ООБД.

Тема 12 Использование XML при работе с БД. ОПК-2.2, ОПК ОС-11.1

XML-ориентированные базы данных. Типы XML-данных в SQL XML-ориентированные базы данных. Типы XML-данных в SQL Server. Импорт и экспорт XML-документов в SQL Server. Компоненты массовой загрузки XML.

4. Типы оценочных материалов, показатели и критерии оценивания

4.1. Оценочные материалы по дисциплине Б1.О.18 «Базы данных» входят в состав оценочных материалов по образовательной программе. Совокупность оценочных материалов по всем дисциплинам (модулям) образовательной программы составляет фонд оценочных средств (далее – ФОС). ФОС используется при проведении текущего контроля успеваемости и промежуточной аттестации обучающихся с целью оценивания достижения обучающимися планируемых результатов обучения.

4.2. ФОС разработан как комплекс проверочных заданий различного типа и уровня сложности, включает критерии и шкалы оценивания, а также «ключи» правильных ответов. ФОС формируется как отдельный документ и хранится в электронном виде, доступ к ФОС предоставлен ограниченному кругу лиц.

4.3. Для самостоятельной работы обучающихся при подготовке к текущему контролю успеваемости и промежуточной аттестации в рабочих программах дисциплин размещены типовые проверочные задания, которые можно условно разделить на задания закрытого, комбинированного и открытого типов.

Задания закрытого типа — это тестовые задания, в которых каждый вопрос сопровождается готовыми вариантами ответов, из которых

необходимо выбрать один или несколько правильных.

Задания комбинированного типа – это тестовые задания, в которых каждый вопрос сопровождается готовыми вариантами ответов, из которых необходимо выбрать один или несколько правильных и обосновать свой выбор.

Задания открытого типа — это задания, в которых на каждый вопрос должен быть предложен развернутый обоснованный ответ.

В зависимости от типа задания рекомендованы определенная последовательность выполнения и система оценивания выполнения заданий.

4.4. Типы заданий, сценарии выполнения, критерии оценивания

ТИП ЗАДАНИЯ	ИНСТРУКЦИЯ	СЦЕНАРИИ ВЫПОЛНЕНИЯ	КРИТЕРИИ ОЦЕНИВАНИЯ
Задание закрытого типа с выбором одного правильного ответа из нескольких предложенных	Прочитайте текст, выберите правильный ответ	1. Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов. 2. Внимательно прочитать предложенные вариант-ты ответа. 3. Выбрать один верный ответ. 4. Записать только номер (или букву) выбранного варианта ответа (например, 3 или В).	Ответ считается верным, если правильно указана цифра или буква
Задание закрытого типа на установление соответствия	Прочитайте текст и установите соответствие	1. Внимательно прочитать текст задания и понять, что в качестве ответа ожидаются пары элементов. 2. Внимательно прочитать оба списка: список 1 – вопросы, утверждения, факты, понятия и т.д.; список 2 – утверждения, свойства объектов и т.д. 3. Сопоставить элементы списка 1 с элементами списка 2, сформировать пары элементов. 4. Записать попарно буквы и цифры (в зависимости от задания) вариантов ответа (например, А1 или Б4).	Ответ считается верным, если правильно указаны цифры или буквы
Задание закрытого типа с выбором нескольких правильных ответов из нескольких	Прочитайте текст, выберите правильные ответы	1. Внимательно прочитать текст задания и понять, что в качестве ответа ожидается несколько правильных ответов из предложенных вариантов.	Ответ считается верным, если правильно установлены все соответствия (позиции из одного столбца верно

ТИП ЗАДАНИЯ	ИНСТРУКЦИЯ	СЦЕНАРИИ ВЫПОЛНЕНИЯ	КРИТЕРИИ ОЦЕНИВАНИЯ
вариантов предложенных		2. Внимательно прочитать предложенные вариант-ты ответа. 3. Выбрать несколько правильных ответов. 4. Записать только номера (или буквы) выбранного варианта ответа (например, 1 4 или А Г).	сопоставлены с позициями другого)
Задание закрытого типа на установление последовательности	Прочитайте текст и установите последовательность	1. Внимательно прочитать текст задания и понять, что в качестве ответа ожидается последовательность элементов. 2. Внимательно прочитать предложенные варианты ответа. 3. Построить верную последовательность из предложенных элементов. 4. Записать буквы/цифры (в зависимости от задания) вариантов ответа в нужной последовательности (например, БВА или 135).	Ответ считается верным, если правильно указана вся последовательность цифр
Задание комбинированного типа с выбором одного правильного ответа из предложенных и обоснованием выбора	Прочитайте текст, выберите правильный ответ и запишите аргументы, обосновывающие выбор ответа	1. Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов. 2. Внимательно прочитать предложенные варианты ответа. 3. Выбрать один верный ответ. 4. Записать только номер (или букву) выбранного варианта ответа.	Ответ считается верным, если правильно указана цифра или буква и приведены корректные аргументы, используемые при выборе ответа

ТИП ЗАДАНИЯ	ИНСТРУКЦИЯ	СЦЕНАРИИ ВЫПОЛНЕНИЯ	КРИТЕРИИ ОЦЕНИВАНИЯ
		5. Записать аргументы, обосновывающие выбор ответа (например, 4 текст обоснования).	
Задание открытого типа с развернутым ответом	Прочитайте текст и запишите развернутый обоснованный ответ	1. Внимательно прочитать текст задания и понять суть вопроса. 2. Продумать логику и полноту ответа. 3. Записать ответ, используя четкие компактные формулировки. 4. В случае расчетной задачи, записать решение и ответ	Ответ считается верным: 1. Отсутствие фактических ошибок. 2. Раскрытие объема используемых понятий (полнота ответа). 3. Обоснованность ответа (наличие аргументов). 4. Логическая последовательность излагаемого материала.

4.5. Общая шкала оценивания результатов текущего контроля успеваемости и промежуточной аттестации обучающихся с применением БРС

Итоговая балльная оценка	Традиционная система	Бинарная система	ECTS	
			Для традиционной системы	Для бинарной системы
95-100	Отлично	Зачтено	A	P/ Passed
85-94			B	P/ Passed
75-84	Хорошо		C	P/ Passed
65-74			D	P/ Passed
55-64	Удовлетворительно		E	P/ Passed
0-54	Неудовлетворительно	Не зачтено	F	F/Failed

Соотношение баллов за текущий контроль успеваемости и промежуточную аттестацию, а также повторную промежуточную аттестацию:

Максимальная сумма баллов за текущий контроль успеваемости	Максимальная сумма баллов за промежуточную аттестацию	Максимальная итоговая балльная оценка	Максимальная сумма баллов за повторную промежуточную аттестацию
60 баллов	40 баллов	100 баллов	100 баллов

5. Формы аттестации, типовые оценочные материалы для текущего контроля успеваемости обучающихся, критерии и шкалы оценивания по контрольным точкам

5.1. В ходе реализации дисциплины Б1.О.18 «Базы данных» используются следующие формы текущего контроля успеваемости обучающихся (в том числе, задания к контрольным точкам): опрос, контрольное задание открытого типа с развернутым ответом.

5.2. Типовые оценочные материалы для текущего контроля успеваемости обучающихся (вне контрольных точек):

Тема 1 Введение в базы данных и SQL ОПК-2.2

Вопросы для опроса

1. Дайте определение понятий «база данных», «предметная область»
2. Каковы предпосылки создания баз данных

3. Определите соотношение понятий «информация» и «данные».
4. Какие технические средства используются для создания баз данных.
5. Дайте определение системе управления базами данных.
6. Определите основные функции и назначение СУБД
7. Перечислите основные категории пользователей баз данных.
8. Дайте определение понятий «база данных», «предметная область»
9. Каковы предпосылки создания баз данных
10. Определите соотношение понятий «информация» и «данные».
11. Какие технические средства используются для создания баз данных.
12. Какие группы операторов выделяются в составе языка SQL?
13. Дайте неформальное определение основных понятий реляционной модели данных: отношение, кортеж, атрибут.
14. Для чего нужны внешние ключи в реляционных таблицах?
15. Что такое потенциальный ключ?

Контрольные задания с развернутым ответом

Задание 1

Предложите пример избыточного потенциального ключа для одной из таблиц базы данных «Авиаперевозки» и объясните, почему он будет избыточным.

Задание 2

В реализации базы данных «Авиаперевозки» предполагается, что самолеты одной модели могут иметь только одну компоновку салона. Представим, что руководством принято решение о том, что нужно учитывать возможность наличия различных компоновок для каждой модели. Какие таблицы придется модифицировать в таком случае и каким образом? Потребуется ли создавать дополнительные таблицы?

Тема 2 Создание рабочей среды ОПК-2.2

Вопросы для опроса

1. Зачем нужен Docker?
2. Что такое PostgreSQL?
3. Зачем использовать PostgreSQL в контейнере?
4. Как подключиться к PostgreSQL в контейнере?

Контрольные задания с развернутым ответом

Задание 1

Выполните процедуру установки СУБД PostgreSQL в среде выбранной вами операционной системы.

Задание 2

Ознакомьтесь с утилитой `psql` с помощью встроенной справки, а также с помощью справки, вызываемой по команде ***psql –help***

Задание 3

Кроме утилиты psql существуют и другие универсальные программы для работы с сервером баз данных PostgreSQL, например, pgAdmin. Это мощная утилита с графическим интерфейсом. Самостоятельно установите программу pgAdmin и изучите основные приемы работы с ней.

Тема 3 Основные операции с таблицами ОПК-2.2

Вопросы для опроса

1. Таблицы в PostgreSQL: типы и операции.
2. Что такое таблицы в PostgreSQL.
3. Обычные таблицы в PostgreSQL.
3. Секционированные таблицы.
4. Временные таблицы PostgreSQL.
5. Системные таблицы PostgreSQL.
6. Широкие таблицы PostgreSQL.
7. Связанные таблицы PostgreSQL.

Контрольные задания с развернутым ответом

Задание 1

Попробуйте ввести в таблицу aircrafts строку с таким значением атрибута «Код самолета» (aircraft_code), которое вы уже вводили, например:

```
INSERT INTO aircrafts  
VALUES ('SU9', 'Sukhoi SuperJet-100', 3000 );
```

Обратите внимание, что в этой команде мы не привели список атрибутов, что вполне допустимо при задании значений атрибутов в том же порядке, в котором атрибуты следуют в определении таблицы. Но в ваших прикладных программах так поступать все же не следует, поскольку в случае возможной реструктуризации таблицы и изменения порядка следования атрибутов в ней ваши команды INSERT могут перестать работать корректно.

Вы получите сообщение об ошибке.

ОШИБКА: повторяющееся значение ключа нарушает ограничение уникальности "aircrafts_pkey"

ПОДРОБНОСТИ: Ключ "(aircraft_code)=(SU9)" уже существует. Подумайте, почему появилось сообщение.

Если вы забыли структуру таблицы aircrafts, то можно вывести ее определение на экран с помощью команды \d aircrafts.

Задание 2

Предложение ORDER BY команды SELECT позволяет отсортировать данные при выводе. По умолчанию сортировка выполняется по возрастанию значений атрибута, указанного в этом предложении. Но можно упорядочить строки и по убыванию значения атрибута. Для этого нужно после имени атрибута в предложении ORDER BY добавить ключевое слово DESC (это сокращение от слова descendant — убывающий порядок). Самостоятельно напишите команду для выборки всех строк из таблицы aircrafts, чтобы

строки были упорядочены по убыванию значения атрибута «Максимальная дальность полета, км» (range).

Задание 3

Команда UPDATE позволяет в процессе обновления выполнять арифметические действия над значениями, находящимися в строках таблицы. Представим себе, что двигатели самолета Sukhoi SuperJet стали в два раза экономичнее, вследствие чего дальность полета этого лайнера возросла ровно в два раза. Команда UPDATE позволяет увеличить значение атрибута range в строке, хранящей информацию об этом самолете, даже не выполняя предварительно выборку с целью выяснения текущего значения этого атрибута. При присваивании нового значения атрибуту range можно справа от знака «=» написать не только числовую константу, но и целое выражение. В нашем случае оно будет простым: `range = range * 2`. Самостоятельно напишите команду UPDATE полностью, при этом не забудьте, что увеличить дальность полета нужно только у одной модели — Sukhoi SuperJet, поэтому необходимо использовать условие WHERE. Затем с помощью команды SELECT проверьте полученный результат.

Задание 4

Если в предложении WHERE команды DELETE вы укажете логически и синтаксически корректное условие, но строк, удовлетворяющих этому условию, в таблице не окажется, то в ответ СУБД выведет сообщение

DELETE 0

Такая ситуация не является ошибкой или сбоем в работе СУБД. Например, если после удаления какой-то строки вы повторно попытаетесь удалить ее же, то получите именно такое сообщение.

Самостоятельно смоделируйте описанную ситуацию, подобрав условие, которому гарантированно не соответствует ни одна строка в таблице «Самолеты» (aircrafts).

Тема 4 Типы данных СУБД PostgreSQL ОПК-2.2

Вопросы для опроса

1. Целочисленные типы в PostgreSQL.
2. Числа с произвольной точностью в PostgreSQL.
3. Типы с плавающей точкой в PostgreSQL.
4. Последовательные типы в PostgreSQL.
5. Символьные типы в PostgreSQL.
6. Шестнадцатеричный формат bytea в PostgreSQL.
7. Формат спецпоследовательностей bytea в PostgreSQL.
8. Типы даты/времени в PostgreSQL.
9. Логический тип в PostgreSQL.
10. Типы перечислений в PostgreSQL.
11. Безопасность типа в PostgreSQL.
12. Геометрические типы: Точки; Прямые; Отрезки; Прямоугольники; Пути; Многоугольники; Окружности в PostgreSQL.

13. Типы, описывающие сетевые адреса: inet; cidr; Различия inet и cidr; macaddr; macaddr8 в PostgreSQL.
14. Битовые строки в PostgreSQL.
15. Типы, предназначенные для текстового поиска: tsvector; tsquery; Тип UUID в PostgreSQL.
16. Тип XML.
17. Типы JSON.
18. Трансформации.
19. Объявления типов массивов; Ввод значения массива; Обращение к массивам; Изменение массивов; Поиск значений в массивах; Синтаксис вводимых и выводимых значений массива в PostgreSQL.
20. Составные типы в PostgreSQL.

Контрольные задания с развернутым ответом

Задание 1

Создайте таблицу, содержащую атрибут типа numeric(precision, scale). Пусть это будет таблица, содержащая результаты каких-то измерений. Команда может быть, например, такой:

CREATE TABLE test_numeric (measurement numeric(5, 2), description text);

Попробуйте с помощью команды INSERT продемонстрировать округление вводимого числа до той точности, которая задана при создании таблицы. Подумайте, какая из следующих команд вызовет ошибку и почему? ***Проверьте свои предположения, выполнив эти команды.***

INSERT INTO test_numeric VALUES (999.9999, 'Какое-то измерение'); INSERT INTO test_numeric

VALUES (999.9009, 'Еще одно измерение'); INSERT INTO test_numeric VALUES (999.1111, 'И еще измерение');

INSERT INTO test_numeric

VALUES (998.9999, 'И еще одно');

Продемонстрируйте генерирование ошибки при попытке ввода числа, количество цифр в котором слева от десятичной точки (запятой) превышает допустимое.

Задание 2

Предположим, что возникла необходимость хранить в одном столбце таблицы данные, представленные с различной точностью. Это могут быть, например, результаты физических измерений разнородных показателей или различные медицинские показатели здоровья пациентов (результаты анализов). В таком случае можно использовать тип numeric без указания масштаба и точности.

Команда для создания таблицы может быть, например, такой: ***CREATE TABLE test_numeric (measurement numeric, description text);***

Если у вас в базе данных уже есть таблица с таким же именем, то можно предварительно ее удалить с помощью команды

DROP TABLE test_numeric;

Вставьте в таблицу несколько строк:

INSERT INTO test_numeric

VALUES (1234567890.0987654321,

'Точность 20 знаков, масштаб 10 знаков');

INSERT INTO test_numeric

VALUES (1.5, 'Точность 2 знака, масштаб 1 знак');

INSERT INTO test_numeric

VALUES (0.12345678901234567890,

'Точность 21 знак, масштаб 20 знаков');

INSERT INTO test_numeric

VALUES (1234567890,

'Точность 10 знаков, масштаб 0 знаков (целое число)');

Теперь сделайте выборку из таблицы и посмотрите, что все эти разнообразные значения сохранены именно в том виде, как вы их вводили.

Задание 3

При работе с числами типов real и double precision нужно помнить, что сравнение двух чисел с плавающей точкой на предмет равенства их значений может привести к неожиданным результатам.

Например, сравним два очень маленьких числа (они представлены в экспоненциальной форме записи):

SELECT '5e-324'::double precision > '4e-324'::double precision;

?column?

----- f

(1 строка)

Чтобы понять, почему так получается, выполните еще два запроса.

SELECT '5e-324'::double precision;

float8

4.94065645841247e-324

(1 строка)

SELECT '4e-324'::double precision;

float8

4.94065645841247e-324

(1 строка)

Самостоятельно проведите аналогичные эксперименты с очень большими числами, находящимися на границе допустимого диапазона для чисел типов real и double precision.

Задание 4

Немного усложним определение таблицы из предыдущего задания. Пусть теперь столбец id будет первичным ключом этой таблицы. CREATE TABLE test_serial (id serial PRIMARY KEY, name text); Теперь выполните следующие команды для добавления строк в таблицу и удаления одной

строки из нее. Для пошагового управления этим процессом выполняйте выборку данных из таблицы с помощью команды SELECT после каждой команды вставки или удаления. INSERT INTO test_serial (name) VALUES ('Вишневая'); Явно зададим значение столбца id: INSERT INTO test_serial (id, name) VALUES (2, 'Прохладная'); При выполнении этой команды СУБД выдаст сообщение об ошибке. Почему? INSERT INTO test_serial (name) VALUES ('Грушевая'); Повторим эту же команду. Теперь все в порядке. Почему? INSERT INTO test_serial (name) VALUES ('Грушевая'); Добавим еще одну строку. INSERT INTO test_serial (name) VALUES ('Зеленая'); А теперь удалим ее же. DELETE FROM test_serial WHERE id = 4; Добавим последнюю строку. INSERT INTO test_serial (name) VALUES ('Луговая'); Теперь сделаем выборку. SELECT * FROM test_serial; Вы увидите, что в нумерации образовалась «дыра». Это из-за того, что при формировании нового значения из последовательности поиск максимального значения, уже имеющегося в столбце, не выполняется.

id	name
1	Вишневая
2	Прохладная
3	Грушевая
5	Луговая

(4 строки)

Задание 6

Типы данных real и double precision поддерживают специальное значение NaN, которое означает «не число» (not a number). В математике существует такое понятие, как неопределенность. В качестве одного из ее вариантов служит результат операции умножения нуля на бесконечность. Посмотрите, что выдаст в результате PostgreSQL:

```
SELECT 0.0 * 'Inf'::real;
```

```
?column?
```

```
----- NaN
```

```
(1 строка)
```

В документации утверждается, что значение NaN считается равным другому значению NaN, а также что значение NaN считается большим любого другого «нормального» значения, т. е. не-NaN. Проверьте эти утверждения с помощью SQL-команды SELECT. Например, сравните значения NaN и Infinity.

```
select 'NaN'::real > 'Inf'::real;
```

```
?column?
```

```
----- t
```

```
(1 строка)
```

Задание 5

Формат ввода и вывода даты можно изменить с помощью параметра datestyle. Значение этого параметра состоит из двух компонентов: первый управляет форматом вывода даты, а второй регулирует порядок следования составных частей даты (год, месяц, день) при вводе и выводе. Текущее значение этого параметра можно узнать с помощью команды SHOW: SHOW datestyle; По умолчанию он имеет такое значение:

```
DateStyle
```

```
-----
```

ISO, DMY

(1 строка)

Продemonстрируем влияние этого параметра на работу с типами date и timestamp. Для экспериментов возьмем дату, в которой число (день) превышает 12, чтобы нельзя было день перепутать с номером месяца. Пусть это будет, например, 18 мая 2016 г.

```
SELECT '18-05-2016'::date;
```

Хотя порядок следования составных частей даты задан в виде «DMY», т. е. «день, месяц, год», но при выводе он изменяется на «год, месяц, день».

```
date  
----- 2016-05-18
```

(1 строка)

Попробуем ввести дату в порядке «месяц, день, год»:

```
SELECT '05-18-2016'::date;
```

В ответ получим сообщение об ошибке. Если бы мы выбрали дату, в которой число (день) было бы не больше 12, например, 9, то сообщение об ошибке не было бы сформировано, т. е. мы с такой датой не смогли бы проиллюстрировать влияние значения «DMY» параметра datestyle. Но главное, что в таком случае мы бы просто не заметили допущенной ошибки.

А вот использовать порядок «год, месяц, день» при вводе можно несмотря на то, что параметр datestyle предписывает «день, месяц, год». Порядок «год, месяц, день» является универсальным, его можно использовать всегда, независимо от настроек параметра datestyle.

```
SELECT '2016-05-18'::date;
```

```
Date  
----- 2016-05-18  
(1 строка)
```

Продолжим экспериментирование с параметром datestyle. Давайте изменим его значение. Сделать это можно многими способами, но мы упомянем лишь некоторые:

- изменив его значение в конфигурационном файле postgresql.conf, который в нашей инсталляции PostgreSQL окружения PGDATESTYLE;
- воспользовавшись командой SET.

Сейчас выберем третий способ, а первые два рассмотрим при выполнении других заданий. Поскольку параметр datestyle состоит фактически из двух частей, которые можно задавать не только обе сразу, но и по отдельности, изменим только порядок следования составных частей даты, не изменяя формат вывода с «ISO» на какой-либо другой.

```
SET datestyle TO 'MDY';
```

Повторим одну из команд, выполненных ранее. Теперь она должна вызвать ошибку. Почему?

```
SELECT '18-05-2016'::date;
```

А такая команда, наоборот, теперь будет успешно выполнена:

```
SELECT '05-18-2016'::date;
```

Теперь приведите настройку параметра `datestyle` в исходное состояние:

```
SET datestyle TO DEFAULT;
```

Самостоятельно выполните команды `SELECT`, приведенные выше, но замените в них тип `date` на тип `timestamp`. Вы увидите, что дата в рамках типа `timestamp` обрабатывается аналогично типу `date`.

Сейчас изменим сразу обе части параметра `datestyle`:

```
SET datestyle TO 'Postgres, DMY';
```

Проверьте полученный результат с помощью команды `SHOW`. Самостоятельно выполните команды `SELECT`, приведенные выше, как для значения типа `date`, так и для значения типа `timestamp`. Обратите внимание, что если выбран формат «Postgres», то порядок следования составных частей даты (день, месяц, год), заданный в параметре `datestyle`, используется не только при вводе значений, но и при выводе. Напомним, что вводом мы считаем команду `SELECT`, а выводом — результат ее выполнения, выведенный на экран.

В документации сказано, что формат вывода даты может принимать значения «ISO», «Postgres», «SQL» и «German». Первые два варианта мы уже рассмотрели. Самостоятельно поэкспериментируйте с двумя оставшимися по той же схеме, по которой вы уже действовали ранее при выполнении этого задания. Можно воспользоваться и стандартными функциями `current_date` и `current_timestamp`.

Задание 6

Представлены описания множества полезных функций, позволяющих преобразовать в строку данные других типов, например, `timestamp`. Одна из таких функций — `to_char()`.

Приведем несколько команд, иллюстрирующих использование этой функции. Ее первым параметром является форматируемое значение, а вторым — шаблон, описывающий формат, в котором это значение будет представлено при вводе или выводе. Сначала попробуйте разобраться, не обращаясь к документации, в том, что означает второй параметр этой функции в каждой из приведенных команд, а затем проверьте свои предположения по документации.

```
SELECT to_char( current_timestamp, 'mi:ss' );
```

```
to_char
```

```
----- 47:43
```

```
(1 строка)
```

```
SELECT to_char( current_timestamp, 'dd' );
```

```
to_char
```

```
-----
```

```
12
```

```
(1 строка)
```

```
SELECT to_char( current_timestamp, 'yyyy-mm-dd' );
```

to_char

2017-03-12

(1 строка)

Поэкспериментируйте с этой функцией, извлекая из значения типа timestamp различные поля и располагая их в нужном вам порядке.

Задание 7

Можно с высокой степенью уверенности предположить, что при прибавлении интервалов к датам и временным отметкам PostgreSQL учитывает тот факт, что различные месяцы имеют различное число дней. Но как это реализуется на практике? Например, что получится при прибавлении интервала в 1 месяц к последнему дню января и к последнему дню февраля? Сначала сделайте обоснованные предположения о результатах следующих двух команд, а затем проверьте предположения на практике и проанализируйте полученные результаты:

```
SELECT ( '2016-01-31'::date + '1 mon'::interval ) AS new_date;  
SELECT ( '2016-02-29'::date + '1 mon'::interval ) AS new_date;
```

Задание 8

Обратимся к таблице, создаваемой с помощью команды

```
CREATE TABLE test_bool ( a boolean, b text );
```

Как вы думаете, какие из приведенных ниже команд содержат ошибку?

```
INSERT INTO test_bool VALUES ( TRUE, 'yes' );  
INSERT INTO test_bool VALUES ( yes, 'yes' );  
INSERT INTO test_bool VALUES ( 'yes', true );  
INSERT INTO test_bool VALUES ( 'yes', TRUE );  
INSERT INTO test_bool VALUES ( '1', 'true' );  
INSERT INTO test_bool VALUES ( 1, 'true' );  
INSERT INTO test_bool VALUES ( 't', 'true' );  
INSERT INTO test_bool VALUES ( 't', truth );  
INSERT INTO test_bool VALUES ( true, true );  
INSERT INTO test_bool VALUES ( 1::boolean, 'true' );  
INSERT INTO test_bool VALUES ( 111::boolean, 'true' );
```

Проверьте свои предположения практически, выполнив эти команды.

Задание 9

Давайте сначала рассмотрим одномерные массивы текстовых значений. Предположим, что пилоты авиакомпании имеют возможность высказывать свои пожелания насчет конкретных блюд, из которых должен состоять их обед во время полета. Для учета пожеланий пилотов необходимо модифицировать таблицу pilots.

```
CREATE TABLE pilots
( pilot_name text,
  schedule integer[],
  meal text[]
);
```

Добавим строки в таблицу:

```
INSERT INTO pilots
VALUES ( 'Ivan', '{ 1, 3, 5, 6, 7 }'::integer[],
        '{ "сосиска", "макароны", "кофе" }'::text[] ),
      ( 'Petr', '{ 1, 2, 5, 7 }'::integer [],
        '{ "котлета", "каша", "кофе" }'::text[] ),
      ( 'Pavel', '{ 2, 5 }'::integer[],
        '{ "сосиска", "каша", "кофе" }'::text[] ),
      ( 'Boris', '{ 3, 5, 6 }'::integer[],
        '{ "котлета", "каша", "чай" }'::text[] );
```

```
INSERT 0 4
```

Обратите внимание, что каждое из текстовых значений, включаемых в литерал массива, заключается в двойные кавычки, а в качестве типа данных указывается text[]. Вот что получилось:

```
SELECT * FROM pilots;
```

pilot_name	schedule	meal
Ivan	{1,3,5,6,7}	{сосиска,макароны,кофе}
Petr	{1,2,5,7}	{котлета,каша,кофе}
Pavel	{2,5}	{сосиска,каша,кофе}
Boris	{3,5,6}	{котлета,каша,чай}

(4 строки)

Давайте получим список пилотов, предпочитающих на обед сосиски:

```
SELECT * FROM pilots WHERE meal[ 1 ] = 'сосиска';
```

pilot_name	schedule	meal
Ivan	{1,3,5,6,7}	{сосиска,макароны,кофе}
Pavel	{2,5}	{сосиска,каша,кофе}

(2 строки)

Предположим, что руководство авиакомпании решило, что пища пилотов должна быть разнообразной. Оно позволило им выбрать свой рацион на каждый из четырех дней недели, в которые пилоты совершают полеты. Для нас это решение руководства выливается в необходимость модифицировать таблицу, а именно: столбец meal теперь будет содержать двумерные массивы. Определение этого столбца станет таким:

```
meal text[][]
```

Задание. Создайте новую версию таблицы и соответственно измените команду INSERT, чтобы в ней содержались литералы двумерных массивов. Они будут выглядеть примерно так:

```
{ { "сосиска", "макароны", "кофе" },
  { "котлета", "каша", "кофе" },
  { "сосиска", "каша", "кофе" },
  { "котлета", "каша", "чай" } }'::text[][]
```

Сделайте ряд выборок и обновлений строк в этой таблице. Для обращения к элементам двухмерного массива нужно использовать два

индекса. Не забывайте, что по умолчанию номера индексов начинаются с единицы.

Задание 10

Изучая приемы работы с типами JSON, можно, как и в случае с массивами, пользоваться способностью команды SELECT обходиться без создания таблиц. Покажем лишь один пример. Для добавления нового ключа и соответствующего ему значения в уже существующий объект, можно воспользоваться оператором «|>»:

```
SELECT '{ "sports": "хоккей" }'::jsonb || '{ "trips": 5 }'::jsonb;
-----
{"trips": 5, "sports": "хоккей"}
(1 строка)
```

Самостоятельно ознакомьтесь с ними, используя описанную технологию работы с командой SELECT.

Тема 5 Основы языка определения данных ОПК-2.2

Вопросы для опроса

1. Значения по умолчанию и ограничения целостности.
2. Создание и удаление таблиц.
3. Модификация таблиц.
4. Представления.
5. Схемы базы данных.

Контрольные задания с развернутым ответом

Задание 1

При использовании значений по умолчанию с ключевым словом DEFAULT возможны и ситуации, когда типичным будет не конкретное значение данных, а способ его получения. Например, если мы захотим фиксировать в каждой строке таблицы «Студенты» имя пользователя базы данных, добавившего эту строку в таблицу, тогда необходимо в определение таблицы добавить еще один столбец. Этот столбец по умолчанию будет получать значение, возвращаемое функцией `current_user`.

```
CREATE TABLE students  
( record_book numeric( 5 ) NOT NULL,  
name text NOT NULL,  
doc_ser numeric( 4 ),  
doc_num numeric( 6 ),  
who_adds_row text DEFAULT current_user, -- добавленный столбец  
PRIMARY KEY ( record_book )  
);
```

Эта функция — `current_user` — будет вызываться не при создании таблицы, а при вставке каждой строки. При этом в команде INSERT не

требуется указывать значение для столбца `who_adds_row`, поскольку функция `current_user` будет вызываться самой СУБД

PostgreSQL: INSERT INTO students (record_book, name, doc_ser, doc_num)

VALUES (12300, 'Иванов Иван Иванович', 0402, 543281);

Давайте пойдем дальше и пожелаем фиксировать не только имя пользователя базы данных, добавившего строку в таблицу, но также и момент времени, когда это было сделано. Самостоятельно внесите модификацию в определение таблицы `students` для решения этой задачи, а затем выполните команду `INSERT` для проверки полученного решения.

Если до выполнения этого упражнения вы еще не ознакомились с командой `ALTER TABLE`, то вместо модифицирования определения таблицы сначала удалите ее, а затем создайте заново:

DROP TABLE students;

CREATE TABLE students ...

Задание 2

Посмотрите, какие ограничения уже наложены на атрибуты таблицы «Успеваемость» (`progress`). Воспользуйтесь командой `\d` утилиты `psql`. А теперь предложите для этой таблицы ограничение уровня таблицы.

В качестве примера рассмотрим такой вариант. Добавьте в таблицу `progress` еще один атрибут — «Форма проверки знаний» (`test_form`), который может принимать только два значения: «экзамен» или «зачет». Тогда набор допустимых значений атрибута «Оценка» (`mark`) будет зависеть от того, экзамен или зачет предусмотрены по данной дисциплине. Если предусмотрен экзамен, тогда допускаются значения 3, 4, 5, если зачет — тогда 0 (не зачтено) или 1 (зачтено).

Не забудьте, что значения `NULL` для атрибутов `test_form` и `mark` не допускаются.

Новое ограничение может быть таким:

ALTER TABLE progress

ADD CHECK

(
(test_form = 'экзамен' AND mark IN (3, 4, 5)
)

OR (test_form = 'зачет'
AND mark IN (0, 1)));

Проверьте, как будет работать новое ограничение в модифицированной таблице `progress`. Для этого выполните команды `INSERT`, как удовлетворяющие ограничению, так и нарушающие его.

В таблице уже было ограничение на допустимые значения атрибута `mark`. Как вы думаете, не будет ли оно конфликтовать с новым ограничением?

Проверьте эту гипотезу. Если ограничения конфликтуют, тогда удалите старое ограничение и снова попробуйте добавить строки в таблицу.

Подумайте, какое еще ограничение уровня таблицы можно предложить для этой таблицы?

Задание 3

В определении таблицы «Успеваемость» (progress) на атрибуты term и mark наложены как ограничения CHECK, так и ограничение NOT NULL. Возникает вопрос: не является ли ограничение NOT NULL избыточным? Ведь в ограничении CHECK явно указаны допустимые значения.

Проверьте гипотезу об избыточности ограничения NOT NULL в данном случае. Для этого модифицируйте таблицу, убрав ограничение NOT NULL, и попробуйте добавить в нее строку с отсутствующим значением атрибута term (или mark).

Задание 4

В определении таблицы «Успеваемость» (progress) для атрибута mark не только задано ограничение CHECK, но и установлено значение по умолчанию с помощью ключевого слова DEFAULT:

```
...  
mark numeric( 1 )  
NOT NULL CHECK ( mark >= 3 AND mark <= 5 )  
DEFAULT 5,  
...
```

Как вы думаете, что будет, если в ограничении DEFAULT мы «случайно» допустим ошибку, написав DEFAULT 6? Если в команде INSERT не указать значение для атрибута mark, то на каком этапе эта ошибка будет выявлена: уже на этапе создания таблицы или только при вставке строки в нее?

Вот эта команда может быть вам полезной для проверки гипотезы, поскольку в ней отсутствует передаваемое значение для атрибута mark:

```
INSERT INTO progress ( record_book, subject, acad_year, term )  
VALUES ( 12300, 'Физика', '2025/2026',1 );
```

Задание 5

В стандарте SQL сказано, что при наличии ограничения уникальности, включающего один или более столбцов, все же возможны повторяющиеся значения этих столбцов в разных строках, но лишь в том случае, если это значения NULL. PostgreSQL придерживается такого же подхода.

Модифицируйте определение таблицы «Студенты» (students), добавив ограничение уникальности по двум столбцам: doc_ser и doc_num. А затем проверьте вышеприведенное утверждение, добавив в таблицу не только строки, содержащие конкретные значения этих двух столбцов, но также и по две строки, имеющие следующие свойства:

- одинаковые значения столбца doc_ser и NULL-значения столбца doc_num;
- NULL-значения столбца doc_num и столбца doc_ser.

Подобные вещи возможны, так как NULL-значения не считаются совпадающими. Это можно проверить с помощью команды

```
SELECT (null = null);
```

Она даст такой результат

```
(m. e. NULL):
```

```
?column?
```

```
-----
```

```
(1 строка)
```

Задание 6

Модифицируйте определения таблиц «Студенты» (students) и «Успеваемость» (progress). В таблице students в качестве первичного ключа назначьте комбинацию атрибутов doc_ser и doc_num, а в таблице progress соответствующим образом измените определение внешнего ключа.

```
CREATE TABLE students  
( record_book numeric( 5 )  
NOT NULL UNIQUE,  
name text NOT NULL,  
doc_ser numeric( 4 ),  
doc_num numeric( 6 ),  
PRIMARY KEY ( doc_ser, doc_num )  
);
```

Обратите внимание, что для атрибутов doc_ser и doc_num можно не указывать ограничение NOT NULL: они входят в состав первичного ключа, а в нем NULL-значения не допускаются, поэтому ограничение NOT NULL фактически подразумевается при включении атрибута в состав первичного ключа.

```
CREATE TABLE progress  
( doc_ser numeric( 4 ),  
doc_num numeric( 6 ),  
subject text NOT NULL,  
acad_year text NOT NULL, term numeric( 1 ) NOT NULL CHECK (  
term = 1 OR term = 2 ),  
mark numeric( 1 ) NOT NULL CHECK ( mark >= 3 AND mark <= 5 )  
DEFAULT 5,  
FOREIGN KEY ( doc_ser, doc_num )  
REFERENCES students ( doc_ser, doc_num )  
ON DELETE CASCADE  
ON UPDATE CASCADE  
);
```

Теперь и первичный, и внешний ключи — составные. Проверьте их действие, добавив несколько строк в каждую таблицу.

Задание 7

В таблице «Студенты» (students) есть текстовый атрибут name, на который наложено ограничение NOT NULL. Как вы думаете, что будет,

если при вводе новой строки в эту таблицу дать атрибуту name в качестве значения пустую строку? Например:

```
INSERT INTO students ( record_book, name, doc_ser, doc_num )  
VALUES ( 12300, ' ', 0402, 543281 );
```

Наверное, проектируя эту таблицу, мы хотели бы все же, чтобы пустые строки в качестве значения атрибута name не проходили в базу данных? Какое решение вы можете предложить? Видимо, нужно добавить ограничение CHECK для столбца name. Если вы еще не изучили команду ALTER TABLE, то удалите таблицу students и создайте ее заново с учетом нового ограничения, а если вы уже познакомились с командой ALTER TABLE, то сделайте так:

```
ALTER TABLE students ADD CHECK ( name <> ' ' );
```

Добавив ограничение, попробуйте теперь вставить в таблицу students строку (row), в которой значение атрибута name было бы пустой строкой (string).

Давайте продолжим эксперименты и предложим в качестве значения атрибута name строку, содержащую сначала один пробел, а потом — два пробела.

```
INSERT INTO students VALUES ( 12346, ' ', 0406, 112233 );  
INSERT INTO students VALUES ( 12347, ' ', 0407, 112234 );
```

Для того чтобы «увидеть» эти пробелы в выборке, сделаем так:

```
SELECT *, length( name ) FROM students;
```

Оказывается, эти невидимые значения имеют ненулевую длину. Что делать, чтобы не допустить таких значений-невидимок? Один из способов: возложить проверку таких ситуаций на прикладную программу. А что можно сделать на уровне определения таблицы students? Какое ограничение нужно предложить? В разделе 9.4 документации «Строковые функции и операторы» есть функция trim. Попробуйте воспользоваться ею. Если вы еще не изучили команду ALTER TABLE, то удалите таблицу students и создайте ее заново с учетом нового ограничения, а если уже познакомились с ней, то сделайте так:

```
ALTER TABLE students ADD CHECK (...);
```

Есть ли подобные слабые места в таблице «Успеваемость» (progress)?

Задание 8

Представления могут быть, условно говоря, вертикальными и горизонтальными. При создании вертикального представления в список его столбцов включается лишь часть столбцов базовой таблицы (таблиц). Например:

```
CREATE VIEW airports_names AS  
SELECT airport_code, airport_name, city  
FROM airports;  
SELECT * FROM airports_names;
```

В горизонтальное представление включаются не все строки базовой таблицы (таблиц), а производится их отбор с помощью фраз WHERE или HAVING.

Например:

```
CREATE VIEW siberian_airports AS  
SELECT * FROM airports  
WHERE city = 'Нижний Новгород' OR city = 'Казань';  
SELECT * FROM siberian_airports;
```

Конечно, вполне возможен и смешанный вариант, когда ограничивается как список столбцов, так и множество строк при создании представления.

Подумайте, какие представления было бы целесообразно создать для нашей базы данных «Авиаперевозки». Необходимо учесть наличие различных групп пользователей, например: пилоты, диспетчеры, пассажиры, кассиры.

Создайте представления и проверьте их в работе.

Задание 9

Предположим, что нам понадобилось иметь в базе данных сведения о технических характеристиках самолетов, эксплуатируемых в авиакомпании. Пусть это будут такие сведения, как число членов экипажа (пилоты), тип двигателей и их количество.

Следовательно, необходимо добавить новый столбец в таблицу «Самолеты» (aircrafts). Дадим ему имя specifications, а в качестве типа данных выберем jsonb. Если впоследствии потребуется добавить и другие характеристики, то мы сможем это сделать, не модифицируя определение таблицы.

```
ALTER TABLE aircrafts ADD COLUMN specifications jsonb;
```

```
ALTER TABLE
```

Добавим сведения для модели самолета Airbus A320-200:

```
UPDATE aircrafts
```

```
SET specifications =
```

```
'{ "crew": 2, "engines":
```

```
{ "type": "IAE V2500",
```

```
"num": 2
```

```
}
```

```
}'::jsonb
```

```
WHERE aircraft_code = '320'; UPDATE 1
```

Посмотрим, что получилось: SELECT model, specifications FROM aircrafts WHERE aircraft_code = '320';

model	specifications
Airbus A320-200	{ "crew": 2, "engines": { "num": 2, "type": "IAE V2500" }}

(1 строка)

Можно посмотреть только сведения о двигателях:

```
SELECT model, specifications->'engines' AS engines  
FROM aircrafts  
WHERE aircraft_code = '320';
```

model	engines
Airbus A320-200	{ "num": 2, "type": "IAE V2500" }

(1 строка)

Чтобы получить еще более детальные сведения, например, о типе двигателей, нужно учитывать, что созданный JSON-объект имеет сложную структуру: он содержит вложенный JSON-объект. Поэтому нужно использовать оператор #> для указания пути доступа к ключу второго уровня.

```
SELECT model, specifications #> '{ engines, type }'  
FROM aircrafts  
WHERE aircraft_code = '320';
```

model	?column?
Airbus A320-200	"IAE V2500"

(1 строка)

Тема 6 Запросы ОПК-2.2

Вопросы для опроса

1. Понятие запроса.
2. DDL-запросы.
3. DML-запросы.
4. Представление, как один из видов запроса.
5. Триггер, как один из видов запроса.
6. Процедура, как один из видов запроса.

Контрольные задания с развернутым ответом

Задание 1

Этот запрос выбирает из таблицы «Билеты» (tickets) всех пассажиров с именами, состоящими из трех букв (в шаблоне присутствуют три символа «_»):

```
SELECT passenger_name  
FROM tickets  
WHERE passenger_name LIKE '___ %';
```

Предложите шаблон поиска в операторе LIKE для выбора из этой таблицы всех пассажиров с фамилиями, состоящими из пяти букв.

Задание 2

Самые крупные самолеты в нашей авиакомпании — это Boeing 777-300. Выяснить, между какими парами городов они летают, поможет запрос:

```
SELECT DISTINCT departure_city, arrival_city  
FROM routes r  
JOIN aircrafts a ON r.aircraft_code = a.aircraft_code  
WHERE a.model = 'Boeing 777-300'
```

ORDER BY 1;

departure_city	arrival_city
Екатеринбург	Москва
Москва	Екатеринбург
Москва	Новосибирск
Москва	Пермь
Москва	Сочи
Новосибирск	Москва
Пермь	Москва
Сочи	Москва

(8 строк)

К сожалению, в этой выборке информация дублируется. Пары городов приведены по два раза: для рейса «туда» и для рейса «обратно». Модифицируйте запрос таким образом, чтобы каждая пара городов была выведена только один раз:

departure_city	arrival_city
Москва	Екатеринбург
Новосибирск	Москва
Пермь	Москва
Сочи	Москва

(4 строки)

Задание 3

Для ответа на вопрос, сколько рейсов выполняется из Москвы в Санкт-Петербург, можно написать совсем простой запрос:

```
SELECT count( *)  
FROM routes  
WHERE departure_city = 'Москва' AND arrival_city = 'Санкт-  
Петербург';  
Count
```

12

(1 строка)

А с помощью какого запроса можно получить результат в таком виде?

departure_city	arrival_city	count
Москва	Санкт-Петербург	12

(1 строка)

Задание 4

Ответить на вопрос о том, каковы максимальные и минимальные цены билетов на все направления, может такой запрос:

```
SELECT f.departure_city, f.arrival_city,  
max( tf.amount ), min( tf.amount )  
FROM flights_v f  
JOIN ticket_flights tf ON f.flight_id = tf.flight_id  
GROUP BY 1, 2  
ORDER BY 1, 2;
```

departure_city	arrival_city	max	min
Абакан	Москва	101000.00	33700.00
Абакан	Новосибирск	5800.00	5800.00
Абакан	Томск	4900.00	4900.00
Анадырь	Москва	185300.00	61800.00
Анадырь	Хабаровск	92200.00	30700.00
...			
Якутск	Мирный	8900.00	8100.00
Якутск	Санкт-Петербург	145300.00	48400.00

(367 строк)

А как выявить те направления, на которые не было продано ни одного билета? Один из вариантов решения такой: если на рейсы, отправляющиеся по какому-то направлению, не было продано ни одного билета, то максимальная и минимальная цены будут равны NULL. Нужно получить выборку в таком виде:

departure_city	arrival_city	max	min
Абакан	Архангельск		
Абакан	Грозный		
Абакан	Кызыл		
Абакан	Москва	101000.00	33700.00
Абакан	Новосибирск	5800.00	5800.00
...			

Модифицируйте запрос, приведенный выше.

Задание 5

В разделе 6.4 мы использовали рекурсивный алгоритм в общем табличном выражении. Изучите этот пример, чтобы лучше понять работу рекурсивного алгоритма:

```
WITH RECURSIVE ranges ( min_sum, max_sum )
AS (
  VALUES( 0, 100000 ),
  ( 100000, 200000 ),
  ( 200000, 300000 )
  UNION ALL
  SELECT min_sum + 100000, max_sum + 100000
  FROM ranges
  WHERE max_sum < ( SELECT max( total_amount ) FROM bookings )
)
SELECT * FROM ranges;
```


min_sum	max_sum	
0	100000	исходные строки
100000	200000	
200000	300000	
100000	200000	результат первой итерации
200000	300000	
300000	400000	
200000	300000	результат второй итерации
300000	400000	
400000	500000	
300000	400000	
400000	500000	
500000	600000	
...		
1000000	1100000	результат (n-3)-й итерации
1100000	1200000	
1200000	1300000	
1100000	1200000	результат (n-2)-й итерации
1200000	1300000	
1200000	1300000	результат (n-1)-й итерации (предпоследней)

(36 строк)

Здесь мы с помощью предложения VALUES специально создали виртуальную таблицу из трех строк, хотя для получения требуемого результата достаточно только одной строки (0, 100000). Еще важно то, что предложение UNION ALL не удаляет строки-дубликаты, поэтому мы можем видеть весь рекурсивный процесс порождения новых строк.

При рекурсивном выполнении запроса

```
SELECT min_sum + 100000, max_sum + 100000
FROM ranges
```

```
WHERE max_sum < ( SELECT max( total_amount ) FROM bookings )
```

каждый раз выполняется проверка в условии WHERE. И на (n-2)-й итерации это условие отсеивает одну строку, т. к. после (n-3)-й итерации значение атрибута max_sum в третьей строке было равно 1 300 000.

Ведь запрос

```
SELECT max( total_amount ) FROM bookings;
```

выдаст значение

```
max
-----
1204500.00
(1 строка)
```

Таким образом, после (n-2)-й итерации во временной области остается всего две строки, после (n-1)-й итерации во временной области остается только одна строка. Заключительная итерация уже не добавляет строк в результирующую таблицу, поскольку единственная строка, поданная

на вход команде SELECT, будет отклонена условием WHERE. Работа алгоритма завершается.

Задание 5.1. Модифицируйте запрос, добавив в него столбец level (можно назвать его и iteration). Этот столбец должен содержать номер текущей итерации, поэтому нужно увеличивать его значение на единицу на каждом шаге. Не забудьте задать начальное значение для добавленного столбца в предложении VALUES.

Задание 5.2. Для завершения экспериментов замените UNION ALL на UNION и выполните запрос. Сравните этот результат с предыдущим, когда мы использовали UNION ALL.

Задание 6

В тексте лекции был приведен запрос, выводящий список городов, в которые нет рейсов из Москвы.

```
SELECT DISTINCT a.city  
FROM airports a  
WHERE NOT EXISTS (  
  SELECT * FROM routes r  
  WHERE r.departure_city = 'Москва'  
  AND r.arrival_city = a.city  
)  
AND a.city <> 'Москва'  
ORDER BY city;
```

Можно предложить другой вариант, в котором используется одна из операций над множествами строк: объединение, пересечение или разность. Вместо знака «?» поставьте в приведенном ниже запросе нужное ключевое слово — UNION, INTERSECT или EXCEPT — и обоснуйте ваше решение.

```
SELECT city  
FROM airports  
WHERE city <> 'Москва' ?  
SELECT arrival_city  
FROM routes  
WHERE departure_city = 'Москва'  
ORDER BY city;
```

Задание 7

В тексте лекции мы рассматривали такой запрос: получить перечень аэропортов в тех городах, в которых больше одного аэропорта.

```
SELECT aa.city, aa.airport_code, aa.airport_name  
FROM (  
  SELECT city, count( *)  
  FROM airports  
  GROUP BY city  
  HAVING count( *) > 1  
) AS a  
JOIN airports
```

```

AS aa
ON a.city = aa.city
ORDER BY aa.city, aa.airport_name;

```

Как вы думаете, обязательно ли наличие функции count в подзапросе в предложении SELECT или можно написать просто SELECT city FROM airports Сначала попробуйте дать ответ теоретически, а потом проверьте вашу гипотезу на компьютере.

Задание 8

Предположим, что департамент развития нашей авиакомпании задался вопросом: каким будет общее число различных маршрутов, которые теоретически можно проложить между всеми городами?

Если в каком-то городе имеется более одного аэропорта, то это учитывать не будем, т. е. маршрутом будем считать путь между городами, а не между аэропортами. Здесь мы используем соединение таблицы с самой собой на основе неравенства значений атрибутов.

```

SELECT count( * )
FROM (
  SELECT DISTINCT city
  FROM airports
) AS a1
JOIN ( SELECT DISTINCT city FROM airports ) AS a2
ON a1.city <> a2.city;

```

```

count
-----
10100
(1 строка)

```

Задание. Перепишите этот запрос с общим табличным выражением.

Тема 7 Изменение данных ОПК-2.2

Вопросы для опроса

1. Вставка строк в таблицы.
2. Обновление строк в таблицах.
3. Удаление строк из таблиц.
4. Перечень основных объектов баз данных.
5. Оператор CREATE.
6. Оператор ALTER.

Контрольные задания с развернутым ответом

Задание 1

Добавьте в определение таблицы aircrafts_log значение по умолчанию current_timestamp и соответствующим образом измените команды INSERT, приведенные в тексте лекции.

Задание 2

В предложении RETURNING можно указывать не только символ «*», означающий выбор всех столбцов таблицы, но и более сложные

выражения, сформированные на основе этих столбцов. В тексте лекции мы копировали содержимое таблицы «Самолеты» в таблицу `aircrafts_tmp`, используя в предложении `RETURNING` именно «*». Однако возможен и другой вариант запроса:

```
WITH add_row AS  
( INSERT INTO aircrafts_tmp  
SELECT * FROM aircrafts  
RETURNING aircraft_code, model, range,  
current_timestamp, 'INSERT'  
)  
INSERT INTO aircrafts_log  
SELECT ? FROM add_row;
```

Что нужно написать в этом запросе вместо вопросительного знака?

Задание 3

В тексте лекции в предложениях `ON CONFLICT` команды `INSERT` мы использовали только выражения, состоящие из имени одного столбца. Однако в таблице «Места» (`seats`) первичный ключ является составным и включает два столбца.

Напишите команду `INSERT` для вставки новой строки в эту таблицу и предусмотрите возможный конфликт добавляемой строки со строкой, уже имеющейся в таблице. Сделайте два варианта предложения `ON CONFLICT`: первый — с использованием перечисления имен столбцов для проверки наличия дублирования, второй — с использованием предложения `ON CONSTRAINT`.

Для того чтобы не изменить содержимое таблицы «Места», создайте ее копию и выполняйте все эти эксперименты с таблицей-копией.

Тема 8 Индексы ОПК-2.2

Вопросы для опроса

1. Что такое индекс?
2. Назовите общие характеристики индекса?
3. Индексы по нескольким столбцам.
4. Уникальные индексы.
5. Индексы на основе выражений.
6. Частичные индексы.

Контрольные задания с развернутым ответом

Задание 1

Предположим, что для какой-то таблицы создан уникальный индекс по двум столбцам: `column1` и `column2`. В таблице есть строка, у которой значение атрибута `column1` равно `ABC`, а значение атрибута `column2` — `NULL`. Мы решили добавить в таблицу еще одну строку с такими же значениями ключевых атрибутов, т. е. `column1` — `ABC`, а `column2` — `NULL`.

Как вы думаете, будет ли операция вставки новой строки успешной или завершится с ошибкой? Объясните ваше решение.

Задание 1

В тексте лекции шла речь о выполнении одной и той же выборки из таблицы «Билеты» (tickets) при наличии индекса по столбцу passenger_name и при его отсутствии. Вы видели, что наличие индекса ускоряет выполнение запроса почти на порядок. Если секундомер в утилите psql выключен, то включите его с помощью команды ***\timing on***

Проведите следующий эксперимент: выполните этот запрос несколько раз подряд при отсутствии индекса, а затем создайте индекс и опять выполните этот запрос несколько раз подряд.

```
SELECT count( *)
```

```
FROM tickets
```

```
WHERE passenger_name = 'IVAN IVANOV';
```

Вы увидите, что время выполнения повторных запросов к таблице сокращается, причем, когда создан индекс, оно сокращается на порядок. Как вы думаете, почему?

Задание 2

Известно, что индекс значительно ускоряет работу, если при выполнении запроса из таблицы отбирается лишь небольшая часть строк. Если же эта доля велика, скажем, половина строк или более, то большого положительного эффекта от наличия индекса уже не будет, а возможно даже, что не будет практически никакого эффекта. Наша задача — проверить это утверждение на практике.

Обратимся к таблице «Перелеты» (ticket_flights). В ней имеется столбец «Класс обслуживания» (fare_conditions), который отличается от остальных тем, что в нем могут присутствовать лишь три различных значения: Comfort, Business и Economy.

Если секундомер в утилите psql выключен, то включите его.

Выполните запросы, подсчитывающие количество строк, в которых атрибут fare_conditions принимает одно из трех возможных значений. Каждый из запросов выполните три-четыре раза, поскольку время может немного изменяться, и подсчитайте среднее время. Обратите внимание на число строк, которые возвращает функция count для каждого значения атрибута. При этом среднее время выполнения запросов для трех различных значений атрибута fare_conditions будет различаться незначительно, поскольку в каждом случае СУБД просматривает все строки таблицы.

```
SELECT count( *)
```

```
FROM ticket_flights
```

```
WHERE fare_conditions = 'Comfort';
```

```
SELECT count( *)
```

```
FROM ticket_flights
```

```
WHERE fare_conditions = 'Business';
```

```
SELECT count( *)
```

FROM ticket_flights

WHERE fare_conditions = 'Economy';

Создайте индекс по столбцу fare_conditions. Конечно, в реальной ситуации такой индекс вряд ли целесообразен, но нам он нужен для экспериментов.

Прodelайте те же эксперименты с таблицей ticket_flights. Будет ли различаться среднее время выполнения запросов для различных значений атрибута fare_conditions? Почему это имеет место?

В завершение этого упражнения отметим, что в случае ошибки планировщика при использовании индекса возможно не только отсутствие положительного эффекта, но и значительный отрицательный эффект.

Тема 9 Транзакция. ОПК-2.2, ОПК ОС-10.2

Вопросы для опроса

1. Общая информация о транзакциях.
2. Уровень изоляции Read Uncommitted.
3. Уровень изоляции Read Committed.
4. Уровень изоляции Repeatable Read.
5. Уровень изоляции Serializable.
6. Блокировки.

Контрольные задания с развернутым ответом

Задание 1.

Транзакции, работающие на уровне изоляции Read Committed, видят только свои собственные обновления и обновления, зафиксированные параллельными транзакциями. При этом нужно учитывать, что иногда могут возникать ситуации, которые на первый взгляд кажутся парадоксальными, но на самом деле все происходит в строгом соответствии с этим принципом.

Воспользуемся таблицей «Самолеты» (aircrafts) или ее копией. Предположим, что мы решили удалить из таблицы те модели, дальность полета которых менее 2 000 км. В таблице представлена одна такая модель — Cessna 208 Caravan, имеющая дальность полета 1 200 км. Для выполнения удаления мы организовали транзакцию. Однако параллельная транзакция, которая, причем, началась раньше, успела обновить таблицу таким образом, что дальность полета самолета Cessna 208 Caravan стала составлять 2 100 км, а вот для самолета Bombardier CRJ-200 она, напротив, уменьшилась до 1 900 км. Таким образом, в результате выполнения операций обновления в таблице по-прежнему присутствует строка, удовлетворяющая первоначальному условию, т. е. значение атрибута range у которой меньше 2000.

Наша задача: проверить, будет ли в результате выполнения двух транзакций удалена какая-либо строка из таблицы.

На первом терминале начнем транзакцию, при этом уровень изоляции Read Committed в команде указывать не будем, т. к. он принят по умолчанию:

BEGIN;

```
SELECT *
FROM aircrafts_tmp
WHERE range < 2000;
```

aircraft_code	model	range
CN1	Cessna 208 Caravan	1200

(1 строка)

```
UPDATE aircrafts_tmp
SET range = 2100
WHERE aircraft_code = 'CN1';
UPDATE 1
UPDATE aircrafts_tmp
SET range = 1900
WHERE aircraft_code = 'CR2';
UPDATE 1
```

На втором терминале начнем вторую транзакцию, которая и будет пытаться удалить строки, у которых значение атрибута range меньше 2000.

```
BEGIN;
```

```
BEGIN
```

```
SELECT *
FROM aircrafts_tmp
WHERE range < 2000;
```

aircraft_code	model	range
CN1	Cessna 208 Caravan	1200

(1 строка)

```
DELETE FROM aircrafts_tmp WHERE range < 2000;
```

Введя команду DELETE, мы видим, что она не завершается, а ожидает, когда со строки, подлежащей удалению, будет снята блокировка. Блокировка, установленная командой UPDATE в первой транзакции, снимается только при завершении транзакции, а завершение может иметь два исхода: фиксацию изменений с помощью команды COMMIT (или END) или отмену изменений с помощью команды ROLLBACK.

Давайте зафиксируем изменения, выполненные первой транзакцией. На первом терминале сделаем так:

```
COMMIT;
COMMIT
```

Тогда на втором терминале мы получим такой результат от команды DELETE:

```
DELETE 0
```

Чем объясняется такой результат? Он кажется нелогичным: ведь команда SELECT, выполненная в этой же второй транзакции, показывала наличие строки, удовлетворяющей условию удаления.

Объяснение таково: поскольку вторая транзакция пока еще не видит изменений, произведенных в первой транзакции, то команда DELETE выбирает для удаления строку, описывающую модель Cessna 208 Caravan, однако эта строка была заблокирована в первой транзакции командой UPDATE. Эта команда изменила значение атрибута range в этой строке.

При завершении первой транзакции блокировка с этой строки снимается (со второй строки — тоже), и команда DELETE во второй транзакции получает возможность заблокировать эту строку. При этом команда DELETE данную строку перечитывает и вновь вычисляет условие WHERE применительно к ней. Однако теперь условие WHERE для данной строки уже не выполняется, следовательно, эту строку удалять нельзя. Конечно, в таблице есть теперь другая строка, для самолета Bombardier CRJ-200, удовлетворяющая условию удаления, однако повторный поиск строк, удовлетворяющих условию WHERE в команде DELETE, не производится.

В результате не удаляется ни одна строка. Таким образом, к сожалению, имеет место нарушение согласованности, которое можно объяснить деталями реализации СУБД. Завершим вторую транзакцию:

END;

COMMIT

Вот что получилось в результате:

SELECT * FROM aircrafts_tmp;

aircraft_code	model	range
773	Boeing 777-300	11100
763	Boeing 767-300	7900
SU9	Sukhoi SuperJet-100	3000
320	Airbus A320-200	5700
321	Airbus A321-200	5600
319	Airbus A319-100	6700
733	Boeing 737-300	4200
CN1	Cessna 208 Caravan	2100
CR2	Bombardier CRJ-200	1900

(9 строк)

Задание .

Модифицируйте сценарий выполнения транзакций: в первой транзакции вместо фиксации изменений выполните их отмену с помощью команды ROLLBACK и посмотрите, будет ли удалена строка и какая конкретно.

Задание 2

Когда говорят о таком феномене, как потерянное обновление, то зачастую в качестве примера приводится операция UPDATE, в которой значение какого-то атрибута изменяется с применением одного из действий арифметики. Например:

UPDATE aircrafts_tmp

SET range = range + 200

WHERE aircraft_code = 'CR2';

При выполнении двух и более подобных обновлений в рамках параллельных транзакций, использующих, например, уровень изоляции Read Committed, будут учтены все такие изменения (что и было показано в тексте лекции). Очевидно, что потерянное обновления не происходит.

Предположим, что в одной транзакции будет просто присваиваться новое значение, например, так:

```
UPDATE aircrafts_tmp  
SET range = 2100  
WHERE aircraft_code = 'CR2';
```

А в параллельной транзакции будет выполняться аналогичная команда:

```
UPDATE aircrafts_tmp  
SET range = 2500  
WHERE aircraft_code = 'CR2';
```

Очевидно, что сохранится только одно из значений атрибута range. Можно ли говорить, что в такой ситуации имеет место потерянное обновление? Если оно имеет место, то что можно предпринять для его недопущения? Обоснуйте ваш ответ.

Тема 10 Повышение производительности. ОПК-2.2, ОПК ОС-10.2, ОПК ОС-11.1

Вопросы для опроса

1. Методы просмотра таблиц.
2. Методы формирования соединений наборов строк.
3. Управление планировщиком.
4. Оптимизация запросов.

Контрольные задания с развернутым ответом

Перед выполнением упражнений нужно восстановить измененные значения параметров:

```
SET enable_hashjoin = on;  
SET  
SET enable_nestloop = on;  
SET
```

1. Как вы думаете, почему при сканировании по индексу оценка стоимости ресурсов, требующихся для выдачи первых результатов, не равна нулю, хотя используется индекс, совпадающий с порядком сортировки?

```
EXPLAIN  
SELECT *  
FROM bookings  
ORDER BY book_ref;
```

QUERY PLAN

```
-----  
Index Scan using bookings_pkey on bookings (cost=0.42..8511.24  
rows=262788 width=21)  
(1 строка)
```

2. Как вы думаете, если в запросе присутствует предложение ORDER BY, и создан индекс по тем столбцам, которые фигурируют в предложении ORDER BY, то всегда ли будет использоваться этот индекс или нет? Почему? Проверьте ваши предположения с помощью команды EXPLAIN.

Задание 1 Самостоятельно выполните команду EXPLAIN для запроса, содержащего общее табличное выражение (CTE). Посмотрите, на каком уровне находится узел плана, отвечающий за это выражение, как он оформляется. Учтите, что общие табличные выражения всегда материализуются, т. е. вычисляются однократно и результат их вычисления сохраняется в памяти, а затем все последующие обращения в рамках запроса направляются уже к этому материализованному результату.

4. Прокомментируйте следующий план, попробуйте объяснить значения всех его узлов и параметров.

EXPLAIN

```
SELECT total_amount  
FROM bookings  
ORDER BY total_amount  
DESC LIMIT 5;
```

QUERY PLAN

```
-----  
Limit (cost=8666.69..8666.71 rows=5 width=6)  
-> Sort (cost=8666.69..9323.66 rows=262788 width=6)  
    Sort Key: total_amount DESC  
    -> Seq Scan on bookings (cost=0.00..4301.88 rows=262788  
        width=6)  
(4 строки)
```

5. В подавляющем большинстве городов только один аэропорт, но есть и такие города, в которых более одного аэропорта. Давайте их выявим.

EXPLAIN

```
SELECT city, count( *)  
FROM airports  
GROUP BY city  
HAVING count( *) > 1;
```

QUERY PLAN

```
-----  
HashAggregate (cost=3.82..4.83 rows=101 width=25)  
Group Key: city  
Filter: (count(*) > 1)  
-> Seq Scan on airports (cost=0.00..3.04 rows=104 width=17)  
(4 строки)
```

Для подсчета числа аэропортов в городах используется последовательное сканирование и формирование хеш-таблицы

(HashAggregate), ключом которой является столбец city. Затем из нее отбираются те записи, значения которых соответствуют условию

Filter: (count(*) > 1)

Как вы думаете, чем можно объяснить, что вторая оценка стоимости в параметре cost для узла Seq Scan, равная 3,04, не совпадает с первой оценкой стоимости в параметре cost для узла HashAggregate?

Задание 2 Выполните команду EXPLAIN для запроса, в котором использована какая-нибудь из оконных функций. Найдите в плане выполнения запроса узел с именем WindowAgg. Попробуйте объяснить, почему он занимает именно этот уровень в плане.

7. Проанализируйте план выполнения операций вставки и удаления строк. Причем сделайте это таким образом, чтобы данные в таблицах фактически изменены не были.

Задание 3 Замена коррелированного подзапроса соединением таблиц является одним из способов повышения производительности.

Предположим, что мы задались вопросом: сколько маршрутов обслуживают самолеты каждого типа? При этом нужно учитывать, что может иметь место такая ситуация, когда самолеты какого-либо типа не обслуживают ни одного маршрута. Поэтому необходимо использовать не только представление «Маршруты» (routes), но и таблицу «Самолеты» (aircrafts).

Это первый вариант запроса, в нем используется коррелированный подзапрос.

EXPLAIN ANALYZE

```
SELECT a.aircraft_code AS a_code, a.model,  
( SELECT count( r.aircraft_code )  
FROM routes r  
WHERE r.aircraft_code = a.aircraft_code  
) AS num_routes  
FROM aircrafts a  
GROUP BY 1, 2  
ORDER BY 3 DESC;
```

А в этом варианте коррелированный подзапрос раскрыт и заменен внешним соединением:

EXPLAIN ANALYZE

```
SELECT a.aircraft_code AS a_code,  
a.model, count( r.aircraft_code  
) AS num_routes  
FROM aircrafts a  
LEFT OUTER JOIN routes r  
ON r.aircraft_code = a.aircraft_code  
GROUP BY 1, 2  
ORDER BY 3 DESC;
```

Причина использования внешнего соединения в том, что может найтись модель самолета, не обслуживающая ни одного маршрута, и если не использовать внешнее соединение, она вообще не попадет в результирующую выборку.

Исследуйте планы выполнения обоих запросов. Попробуйте найти объяснение различиям в эффективности их выполнения. Чтобы получить усредненную картину, выполните каждый запрос несколько раз. Поскольку таблицы, участвующие в запросах, небольшие, то различие по абсолютным затратам времени выполнения будет незначительным. Но если бы число строк в таблицах было большим, то экономия ресурсов сервера могла оказаться заметной.

Предложите аналогичную пару запросов к базе данных «Авиаперевозки». Проведите необходимые эксперименты с вашими запросами.

9. Одним из способов повышения производительности является изменение схемы данных, связанное с денормализацией, а именно: создание материализованных представлений. В теме 5 было описано такое материализованное представление — «Маршруты» (routes). Команда для его создания была приведена в теме 6.

Проведите эксперимент: сначала выполните выборку из готового представления, а затем ту выборку, которая это представление формирует.

EXPLAIN ANALYZE

SELECT *

FROM routes;

EXPLAIN ANALYZE

WITH f3 AS (SELECT f2.flight_no, ...

Поскольку второй запрос очень громоздкий, то можно поступить таким образом: сначала сохраните его в текстовом файле, а затем выполните с помощью команды \i утилиты psql.

Вы увидите, что затраты времени отличаются практически на два порядка. Конечно, нужно помнить, что материализованные представления необходимо периодически обновлять, чтобы их содержимое было актуальным.

10. Одним из способов повышения производительности является изменение схемы данных, связанное с денормализацией, а именно: использование вычисляемых столбцов. Для примера рассмотрим таблицу «Бронирования» (bookings). В ней столбец «Полная сумма бронирования» (total_amount) является вычисляемым. Мы не будем сейчас говорить о том, каким образом его значения синхронизируются с данными в таблице «Перелеты» (ticket_flights), а лишь рассмотрим два запроса, возвращающие полные суммы бронирований. Предположим, что указанного столбца в таблице bookings не было бы. Тогда запрос, возвращающий полные суммы бронирований, выглядел бы так:

EXPLAIN ANALYZE

```
SELECT b.book_ref, sum( tf.amount )  
FROM bookings b, tickets t, ticket_flights tf  
WHERE b.book_ref = t.book_ref  
AND t.ticket_no = tf.ticket_no  
GROUP BY 1  
ORDER BY 1;
```

Но благодаря наличию вычисляемого столбца total_amount те же сведения можно получить с гораздо меньшими затратами ресурсов:

```
EXPLAIN ANALYZE  
SELECT book_ref, total_amount  
FROM bookings  
ORDER BY 1;
```

Попробуйте предложить еще какой-нибудь вычисляемый столбец для одной из таблиц базы данных «Авиаперевозки». Проведите эксперименты, подтверждающие эффективность вашего решения.

11. Одним из способов повышения производительности является изменение схемы данных, а именно: использование временных таблиц. Предположим, что нам предстоит сделать много выборок из представления «Рейсы» (flights_v), в таком случае имеет смысл подумать о создании временной таблицы:

```
CREATE TEMP TABLE flights_tt AS  
SELECT * FROM flights_v;
```

Сформируйте планы для получения простой выборки из представления и из временной таблицы и сравните полученные результаты. Как вы думаете, почему план, сформированный для получения даже простой выборки из представления, многоуровневый?

```
EXPLAIN ANALYZE  
SELECT *  
FROM flights_v;  
EXPLAIN ANALYZE  
SELECT *  
FROM flights_tt;
```

Выполните более сложные запросы к представлению и временной таблице и сравните полученные результаты. Чтобы увидеть фактические затраты времени, включайте в команду EXPLAIN опцию ANALYZE.

Подумайте, при выполнении каких запросов к базе данных «Авиаперевозки» было бы целесообразно создать временную таблицу. Создайте ее и проведите эксперименты, подтверждающие эффективность ее использования.

12. Одним из способов повышения производительности является изменение схемы данных, связанное с денормализацией, а именно: создание индексов. Выполните следующий простой запрос к таблице «Билеты»:

```
EXPLAIN ANALYZE  
SELECT count( * )
```

FROM tickets

WHERE passenger_name = 'IVAN IVANOV';

Создайте индекс по столбцу passenger_name:

CREATE INDEX passenger_name_key

ON tickets (passenger_name);

Теперь повторите запрос и сравните полученные планы и фактические результаты.

Предложите какой-нибудь запрос к базе данных «Авиаперевозки», для выполнения которого было бы целесообразно создать индекс. Создайте индекс и повторите запрос. Изучите полученный план, посмотрите, используется ли индекс планировщиком.

13. В самом конце лекции мы выполняли оптимизацию запроса путем создания индекса и модификации текста запроса. Был сформирован такой запрос:

EXPLAIN ANALYZE

SELECT num_tickets, count(*) AS num_bookings

FROM

(SELECT b.book_ref, count(*)

FROM bookings b, tickets t

WHERE date_trunc('mon', b.book_date) = '2016-09-01'

AND t.book_ref = b.book_ref

GROUP BY b.book_ref

) AS count_tickets(book_ref, num_tickets)

GROUP by num_tickets

ORDER BY num_tickets DESC;

Мы экспериментировали с параметрами планировщика enable_hashjoin и enable_nestloop при наличии индекса по таблице tickets.

SET enable_hashjoin = off;

SET enable_nestloop = off;

Однако полученные планы детально рассмотрены не были.

Тема 11 Объектно-ориентированные базы данных ОПК-2.2, ОПК ОС-10.2, ОПК ОС-11.1

Вопросы для опроса

1. Понятие объектно-ориентированных баз данных (ООБД) и объектно-ориентированных систем управления базами данных (ОСУБД).

2. Характеристики ООБД.

3. Достоинства и недостатки модели ООБД.

Контрольные задания с развернутым ответом

Практическая работа Связь СУБД PostgreSQL и Python

Цель работы: произвести связь базы данных в PostgreSQL и Python, изучить операции по манипулированию с данными БД, а также созданию простейших пользовательских функций.

Библиотеки, которые нужны в Python:

- psycopg2 для соединения с БД
- pandas для работы с данными

По желанию вы можете использовать другие библиотеки.

1 Соединение Python с БД в PostgreSQL

Стандартный вариант соединения Python с БД в PostgreSQL:

```
import pandas as pd import psycopg2
```

```
# Подключение к базе данных:
```

```
connection = psycopg2.connect(database="database1", # название базы  
данных
```

```
user="postgres",  
password="admin",  
host="127.0.0.1",  
port="5432")
```

С помощью метода connect() создается подключение к экземпляру
базы данных

PostgreSQL. Здесь

- database – название БД, к которой нужно подключиться,
- user – имя пользователя для аутентификации,
- password – пароль БД,
- host – адрес сервера БД,
- port – номер порта (значение по умолчанию 5432).

С базой данных можно взаимодействовать с помощью класса cursor.

Его можно

получить из метода cursor(), который есть у объекта соединения. Он
поможет выполнять SQL-команды из Python.

Из одного объекта соединения можно создавать неограниченное
количество объектов cursor. Они не изолированы, поэтому любые
изменения, сделанные в базе данных с помощью одного объекта, будут
видны остальным.

С помощью connection.get_dsn_parameters() можно вывести
параметры соединения.

С помощью метода execute объекта cursor можно выполнить любую
операцию

или запрос к базе данных. В качестве параметра этот метод принимает
SQL-запрос. Результаты запроса можно получить с помощью fetchone(),
fetchmany(), fetchall().

Необходимо использовать cursor.close() и connection.close(), так как
правильно всегда закрывать объекты cursor и connection после завершения
работы, чтобы избежать проблем с базой данных.

Выведем данные о соединении и версии СУБД:

```
cursor = connection.cursor()# курсор для выполнения операций с БД  
print(connection.get_dsn_parameters(), "\n") # вывод свойства  
соединения
```

```

cursor.execute("SELECT version();") # выполнение запроса к БД
version_ps = cursor.fetchone() # получение результата запроса
print("Выподключены - ", version_ps, "\n")

```

Результат:

```

{'user': 'postgres', 'channel_binding': 'prefer', 'dbname': 'database1', 'host':
'127.0.0.1', 'port': '5432', 'options': '', 'sslmode': 'prefer', 'sslcompression': '0',
'sslsni': '1', 'ssl_min_protocol_version': 'TLSv1.2', 'gssencmode': 'disable',
'krbsrvname': 'postgres', 'target_session_attrs': 'any'}

```

Вы подключены к - ('PostgreSQL 13.5, compiled by Visual C++ build 1914, 64-bit',)

2 Выполнение запроса SELECT

Выполним запрос на выборку данных из таблицы с работами: sql = "SELECT * FROM jobs " # запрос SQL

```

df = pd.read_sql_query("SELECT * FROM jobs ", connection) print(df)

```

В данном примере использовали read_sql_query библиотеки pandas, который возвращает DataFrame, соответствующий результирующему набору строки запроса.

3 Создание новой таблицы с помощью Python и заполнение ее данными

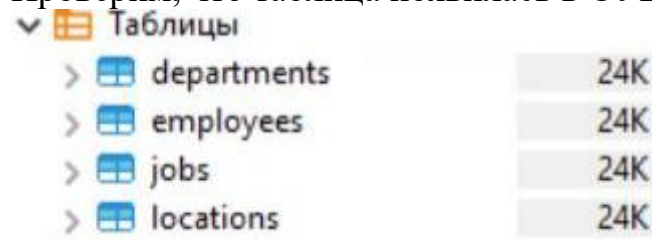
Создадим новую таблицу locations

```

cursor.execute("CREATE TABLE if not exists locations (location_id int
PRIMARY KEY, city varchar(30),
postal_code varchar(12) );")

```

Проверим, что таблица появилась в СУБД PostgreSQL (рисунок 1):



Таблицы	
>	departments 24K
>	employees 24K
>	jobs 24K
>	locations 24K

Рисунок 1 – Создание новой таблицы

Заполним таблицу данными и выведем результат:

```

cursor.execute(
"INSERT INTO locations VALUES (1, 'Roma', '00989')
")
'''

```

```

connection.commit()

```

2

```

table_locations = pd.read_sql_query("SELECT * FROM locations ",
connection)

```

```

print(table_locations)

```

Проверим, что таблица также заполнена в СУБД PostgreSQL (рисунок 2):

	location_id	city	postal_code
1	1	Roma	00989

Рисунок 2 – Заполнение таблицы

4 Создание простейших пользовательских функций в PostgreSQL и Python

Создадим функцию select_data в PostgreSQL, которая на вход получает код отдела и выводит информацию из таблицы departments, фильтруя данные по коду отдела.

```
CREATE FUNCTION select_data(id_dept int) RETURNS
SETOF departments AS $$ SELECT * FROM departments WHERE departm
ents.department_id > id_dept;
```

```
$$ LANGUAGE SQL;
```

```
SELECT * FROM select_data(30);
```

Результат вызова на рисунке 3 (выведены только отделы с id>30):

department_id	department_name	manager_id
50	Human Resources	53
50	Shipping	22
60	IT	4
80	Sales	46
90	Executive	1
100	Finance	9

Рисунок 3 – Создание функции

Теперь вернемся в Python и попробуем вызвать созданную функцию из скрипта в Python:

```
cursor.callproc('select_data',[20,])# вызов функции (название
изPostgreSQL )
```

```
result = cursor.fetchall() # получение результатов
```

```
result_proc = pd.DataFrame(result)# создание датафрейма с
результатом
```

```
print(result_proc)
```

Попробуем выполнить обратное действие, создадим функцию select_data1, аналогичную функции выше в скрипте Python и внесем изменения в БД в PostgreSQL:

```
| #код функции
```

```
postgresql_func = """
```

```
CREATE OR REPLACE FUNCTION select_data1(id_dept int)
```

```
RETURNS SETOF departments AS $$
```

```
SELECT * FROM departments WHERE departments.department_id >
id_dept;
```

```
$$ LANGUAGE SQL;
```

```
"""
```

```
cursor.execute(postgresql_func) # выполнение запроса
```

```
connection.commit() #внесениеизме
connectionзакрытсоед.close()инения
```

#

```
cursorзакрытие.cursorclose() #
```

Проверим, как это выглядит в нашей СУБД (рис.4):

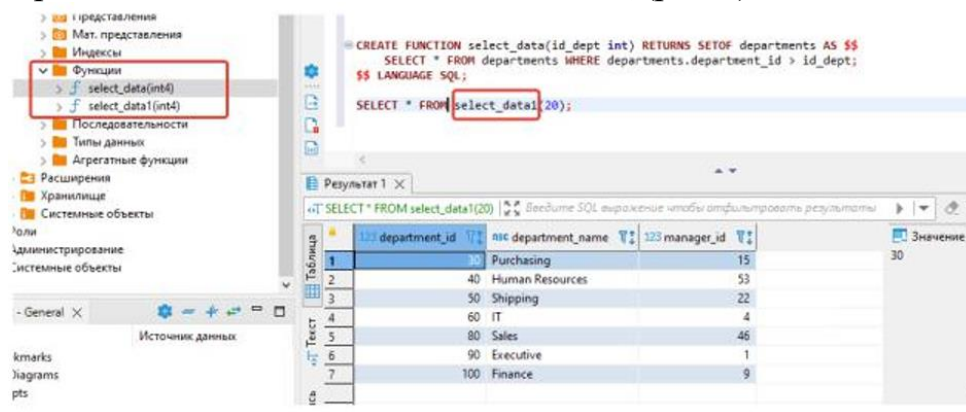


Рисунок 4 - Результат

Порядок выполнения работы:

- 1.Использовать БД, созданную в предыдущей ЛР (employees, departments, jobs). Обязательно предоставить схему данных в отчете.
- 2.Осуществить связь Python и БД в PostgreSQL.
- 3.Создать новую таблицу locations, по примеру методических указаний.

Заполнить таблицу данными:

```
INSERT INTO locations VALUES ( 1,'Roma', '00989'); INSERT
INTO locations VALUES ( 2,'Venice','10934'); INSERT
INTO locations VALUES ( 3,'Tokyo', '1689'); INSERT
INTO locations VALUES ( 4,'Hiroshima','6823'); INSERT
INTO locations VALUES ( 5,'Southlake', '26192');
INSERT INTO locations VALUES ( 6,'South San
Francisco', '99236'); INSERT INTO locations VALUES ( 7,'South
Brunswick','50090'); INSERT INTO locations VALUES ( 8,'Seattle','98199');
INSERT INTO locations VALUES ( 9,'Toronto','M5V 2L7'); INSERT
INTO locations VALUES ( 10,'Whitehorse','YSW 9T2');
```

- 4.Добавить в таблицу с сотрудниками location_id, добавить связь с таблицей locations, заполнить столбец location_id данными. В отчете описать, каким способом было выполнено задание, приложить скриншоты результатов, в том числе обновленную схему данных.

- 5.Выполнить 3 запроса SELECT в Python, предоставить скриншоты результатов и текстовое пояснение:

6. Создать функцию select_data в СУБД PostgreSQL (см. Методические указания 4 пункт), продемонстрировать результат работы. Вызвать функцию select_data в Python, продемонстрировать результат. Создать функцию select_data1 в скрипте Python, продемонстрировать результат вызова этой функции в СУБД PostgreSQL.

Контрольное задание

Создать собственную пользовательскую функцию в СУБД PostgreSQL, используя более сложные SQL-запросы (например, с применением агрегатных функций и связью нескольких таблиц), продемонстрировать результат работы. Создать эту же функцию через скрипт Python.

Таблица 1 - Варианты заданий

Варианты:	Формулировка запросов
1	- Найти сотрудника, фамилия которого состоит более чем из одного слова. - Найти минимальные зарплаты по отделам, но вывести только те, которые больше 10000.
2	- Вывести количество сотрудников в каждом отделе и отсортировать по убыванию числа сотрудников. - Найти сотрудника, работающего программистом (job_id = 'IT_PROG') и имеющего самую высокую зарплату среди коллег.
3	- Найти сотрудника с именем John, имеющего зарплату больше 12000. - Вывести название отдела с максимальным числом сотрудников.
4	- Найти средние зарплаты по каждому из отделов (можно ли как-то округлить полученные значения?). - Вывести названия должностей и количество сотрудников, соответствующих им. - Отсортировать данные по убыванию числа сотрудников.
5	- Найти сотрудника, который имеет зарплату равную минимальной зарплате по его должности. - Найти сотрудника с минимальной зарплатой среди всех сотрудников. Имя и фамилию вывести в одной колонке, дать имя колонке «worker».
6	- Найти первых трёх сотрудников с наименьшей разницей между их зарплатой и минимальной зарплатой по должности. - Найти самое популярное имя (first_name).
7	- Найти сотрудника со второй по счёту минимальной зарплатой. - Выяснить, сколько фондовых менеджеров (Stock_manager) работают в отделе перевозок (Shipping).

Варианты:	Формулировка запросов
8	- Найти количество сотрудников, в должности которых фигурирует слово «Manager». - Найти разницу между зарплатой начальников и средней зарплатой их подчиненных.
9	- Выяснить, сотрудники какой профессии получают зарплату меньше 2500. - Вывести сотрудников и их начальников (в одном столбце расположены сотрудники, во втором – их начальники).
10	- Найти профессию, диапазон которой между минимальной и максимальной зарплатой меньше, чем у остальных профессий. - Вывести названия профессий(job_title) и среднюю зарплату (в диапазоне от 2000 до 5000) сотрудников этих профессий.

Тема 12 Использование XML при работе с БД. ОПК-2.2, ОПК ОС-11.1

Вопросы для опроса

1. XML-ориентированные базы данных.
2. Типы XML-данных в SQL Server.
3. Импорт и экспорт XML-документов в SQL Server.

Контрольные задания с развернутым ответом

Практическая работа. Работа с данными в формате XML

Практическая работа посвящена вопросам обработки данных в формате XML средствами, предоставляемыми СУБД SQL Server. При выполнении этой работы будет использоваться БД ExampleBase, файл с резервной копией которой находится в папке с файлами для данной работы. Получить из резервной копии базу данных можно из Management Studio, выбрав пункт "Восстановить базу данных" в контекстном меню узла "Базы данных" (рис. 1) При необходимости воспользуйтесь справочной системой.

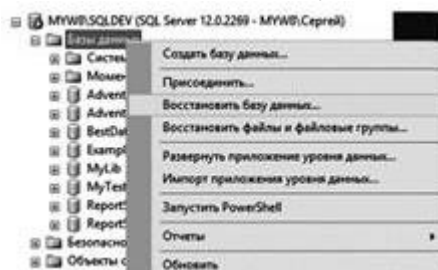


Рис. 1. Восстановление БД

База содержит две таблицы, в первой из которых (Country) – информация о странах, во второй – о регионах (Region). На рис. 2 представлена диаграмма с изображением этих таблиц.



Рис. 2. Диаграмма с таблицами базы ExampleBase

Запустите приложение Management Studio, подключитесь к экземпляру SQL Server и восстановите базу ExampleBase из резервной копии. Ознакомьтесь с содержимым таблиц базы данных. Выполните следующие задания.

Задание 1.

Напишите запрос, выводящий из таблицы Country данные о буквенном коде страны (ID), ее сокращенном и полном названии, столице в виде XML. При этом, корневой элемент должен называться Countries, элемент следующего уровня – Country. Значения столбцов таблицы выводятся как текстовые данные элементов, названия которых соответствуют названиям столбцов. Фрагмент подобного XML-документа приведен ниже:

```

<Countries>
<Country>
<ID> AUS</ID>
<NAME> Австралия </NAME>
<FULLNAME> Австралийский Союз </FULLNAME>
<CAPITAL> Канберра </CAPITAL>
</Country>
</Countries>
    
```

Задание 2

Создайте временную таблицу #T1 со столбцами код страны (первичный ключ), полное и сокращенное название страны, название столицы, перечень регионов (в виде xml-документа). Сначала заполните в ней столбцы, кроме последнего, данными из таблицы Country. Вторым выражением SQL заполните значение последнего столбца (xml), записав туда документ, содержащий перечень регионов соответствующей страны. Название региона оформляйте как текстовые данные элемента Region, код региона (REGID) и название столицы региона оформите как атрибуты соответствующего элемента.

Задание 3

Используя данные из таблицы #T1 и возможности языков SQL и XQuery, напишите запросы, выводящие в реляционном представлении (в виде таблицы):

- название страны и региона, административный центр которого город BELFAST;

- перечень регионов и их административные центры в Австралии (полное название Австралийский Союз) и Великобритании (Соединенное Королевство Великобритании и Северной Ирландии).

5.3. Один или несколько тематических блоков дисциплины завершаются контрольной точкой (далее – КТ). Текущий контроль успеваемости по дисциплине предусматривает не менее 2 (двух) и не более 10 (десяти) КТ в течение периода освоения дисциплины.

Максимальное количество баллов за любой тип работ в рамках КТ составляет 100 (сто) баллов.

Распределение весовых коэффициентов по КТ в рамках текущего контроля успеваемости по дисциплине и формулы расчета:

Наименование контрольной точки	Максимальное количество баллов за работу в рамках КТ, которое может набрать студент	Коэффициент веса контрольной точки	Результат контрольной точки, участвующий в формировании итоговой балльной оценки по дисциплине (отражается в журнале БРС в СДО)
КТ 1	100	0,2	20
КТ 2	100	0,2	20
КТ 3	100	0,2	20
Итого:	х	0,6	60
КТ 4	100	0,2	20
КТ 5	100	0,2	20
КТ6	100	0,2	20
Итого:	х	0,6	60

Формула расчета результата контрольной точки:

Результат контрольной точки = Количество баллов за работу в рамках КТ х Коэффициент веса контрольной точки.

5.4. Формы текущего контроля успеваемости обучающихся в рамках КТ и типовые оценочные материалы:

КТ – 1.

Тема 1 и 2

Опрос:

Вопросы для опроса:

№ п.п.	Содержание вопроса
1.	Что такое база данных?
2.	Какие типы баз данных наиболее распространены в практике?
3.	Что такое первичный ключ в реляционной базе данных?

№ п.п.	Содержание вопроса
4.	Для чего нужны внешние ключи в реляционных таблицах?
5.	Что такое потенциальный ключ?
6.	Что подразумевается под таблицей и полем в SQL?
7.	Что такое соединения в SQL?
8.	Что такое уникальный ключ (Unique key)?
9.	Что такое внешний ключ?
10.	Как установить PostgreSQL на выбранной операционной системе?
11.	Как выбрать порт для службы PostgreSQL
12.	Как настроить конфигурационные файлы PostgreSQL?
13.	Как изменить строку listen_addresses в postgresql.conf
14.	Как настроить фильтрацию по IP-адресам
15.	Как использовать SSL

Критерии оценивания опроса:

Диапазон баллов	Описание критерия
85-100	Обучающийся полно излагает материал (отвечает на вопрос), дает правильное определение основных понятий; обнаруживает понимание материала, может обосновать свои суждения, применить знания на практике, привести необходимые примеры не только из учебника, но и самостоятельно составленные; технически излагает материал.
65-84	Обучающийся дает ответ, удовлетворяющий тем же требованиям, что и для оценки «отлично», но допускает 1–2 ошибки, которые сам же исправляет, и 1–2 недочета в последовательности и языковом оформлении излагаемого.
55-64	Обучающийся обнаруживает знание и понимание основных положений данной темы, но излагает материал неполно и допускает неточности в определении понятий или формулировке правил; не умеет достаточно глубоко и доказательно обосновать свои суждения и привести свои примеры; излагает материал непоследовательно и допускает ошибки в языковом оформлении излагаемого.
0-54	Обучающийся обнаруживает незнание вопроса, допускает ошибки в формулировке определений и правил, искажающие их смысл, беспорядочно и неуверенно излагает материал.

КТ – 2.**Тема 3 и 4**Опрос:

Вопросы для опроса:

№ п.п.	Содержание вопроса
1.	Как создать новую таблицу в PostgreSQL с помощью команды CREATE TABLE?
2.	Какие параметры команды позволяют определять столбцы и их атрибуты (например, первичный ключ, ограничения)?
3.	Как создать таблицу, наследующую столбцы из существующей таблицы (с помощью оператора INHERITS)?
4.	Как добавить данные в таблицу с помощью команды INSERT INTO ... VALUES?
5.	Как использовать оператор UPDATE для изменения существующих данных в таблице?
6.	Как использовать подзапросы в UPDATE для назначения значений на основе данных из других таблиц.
7.	Как использовать оператор DELETE для удаления одной или нескольких записей из таблицы?
8.	Какие типы данных для хранения чисел в PostgreSQL?
9.	Какие типы данных для хранения текстовой информации в PostgreSQL?
10.	Какие типы данных для работы с датами и временем в PostgreSQL?
11.	Какие значения может принимать логический тип данных в PostgreSQL?

Критерии оценивания опроса:

Диапазон баллов	Описание критерия
85-100	Обучающийся полно излагает материал (отвечает на вопрос), дает правильное определение основных понятий; обнаруживает понимание материала, может обосновать свои суждения, применить знания на практике, привести необходимые примеры не только из учебника, но и самостоятельно составленные; технически правильно излагает материал.
65-84	Обучающийся дает ответ, удовлетворяющий тем же требованиям, что и для оценки «отлично», но допускает 1–2 ошибки, которые сам же исправляет, и 1–2 недочета в последовательности и языковом оформлении излагаемого.

Диапазон баллов	Описание критерия
55-64	Обучающийся обнаруживает знание и понимание основных положений данной темы, но излагает материал неполно и допускает неточности в определении понятий или формулировке правил; не умеет достаточно глубоко и доказательно обосновать свои суждения и привести свои примеры; излагает материал непоследовательно и допускает ошибки в языковом оформлении излагаемого.
0-54	Обучающийся обнаруживает незнание вопроса, допускает ошибки в формулировке определений и правил, искажающие их смысл, беспорядочно и неуверенно излагает материал.

КТ – 3.

Тема 5-6

Опрос:

Вопросы для опроса:

№ п.п.	Содержание вопроса
1.	Какие группы операторов выделяются в составе языка SQL?
2.	Дайте неформальное определение основных понятий реляционной модели данных: отношение, кортеж, атрибут.
3.	Для чего нужны внешние ключи в реляционных таблицах?
4.	Что такое потенциальный ключ?
5.	Объяснить разницу между операторами DELETE, TRUNCATE и DROP в SQL
6.	Объяснить назначение предложения GROUP BY и как оно работает с агрегатными функциями.
7.	Какова роль файла pg_hba.conf в PostgreSQL?
8.	Как проверить версию PostgreSQL?
9.	Объяснить процесс настройки базы данных PostgreSQL с нуля.
10.	Как работает планировщик запросов PostgreSQL и как на него можно повлиять?
11	Описать роль pg_stat_statements в мониторинге производительности

Критерии оценивания опроса:

Диапазон баллов	Описание критерия
85-100	Обучающийся полно излагает материал (отвечает на вопрос), дает правильное определение основных понятий; обнаруживает понимание материала, может обосновать свои суждения, применить знания на практике, привести необходимые примеры не только из учебника, но и

Диапазон баллов	Описание критерия
	самостоятельно составленные; технически правильно излагает материал.
65-84	Обучающийся дает ответ, удовлетворяющий тем же требованиям, что и для оценки «отлично», но допускает 1–2 ошибки, которые сам же исправляет, и 1–2 недочета в последовательности и языковом оформлении излагаемого.
55-64	Обучающийся обнаруживает знание и понимание основных положений данной темы, но излагает материал неполно и допускает неточности в определении понятий или формулировке правил; не умеет достаточно глубоко и доказательно обосновать свои суждения и привести свои примеры; излагает материал непоследовательно и допускает ошибки в языковом оформлении излагаемого.
0-54	Обучающийся обнаруживает незнание вопроса, допускает ошибки в формулировке определений и правил, искажающие их смысл, беспорядочно и неуверенно излагает материал.

КТ – 4.

Тема 7-8

Опрос:

Вопросы для опроса:

№ п.п.	Содержание вопроса
1.	Как оператор UPDATE используется для изменения существующих данных в таблице?
2.	Какой синтаксис оператора UPDATE?
3.	Какие примеры запросов на изменение данных с помощью оператора UPDATE?
4.	Какие ошибки, возникающие при использовании оператора UPDATE, и как их решать?
5.	Зачем нужны индексы?
6.	Как работают индексы?
7.	Какой тип индекса используется по умолчанию?
8.	Для каких задач подходят другие типы индексов?
9.	Как создать индекс в PostgreSQL?
10.	Как создать уникальный индекс?
11.	Как индексировать часто используемые поля в WHERE, JOIN и ORDER BY?

Критерии оценивания опроса:

Диапазон баллов	Описание критерия
85-100	Обучающийся полно излагает материал (отвечает на вопрос), дает правильное определение основных понятий; обнаруживает понимание материала, может обосновать свои суждения, применить знания на практике, привести необходимые примеры не только из учебника, но и самостоятельно составленные; технически правильно излагает материал.
65-84	Обучающийся дает ответ, удовлетворяющий тем же требованиям, что и для оценки «отлично», но допускает 1–2 ошибки, которые сам же исправляет, и 1–2 недочета в последовательности и языковом оформлении излагаемого.
55-64	Обучающийся обнаруживает знание и понимание основных положений данной темы, но излагает материал неполно и допускает неточности в определении понятий или формулировке правил; не умеет достаточно глубоко и доказательно обосновать свои суждения и привести свои примеры; излагает материал непоследовательно и допускает ошибки в языковом оформлении излагаемого.
0-54	Обучающийся обнаруживает незнание вопроса, допускает ошибки в формулировке определений и правил, искажающие их смысл, беспорядочно и неуверенно излагает материал.

КТ – 5.

Тема 9-10

Опрос:

Вопросы для опроса:

№ п.п.	Содержание вопроса
1.	Что такое транзакция в PostgreSQL
2.	Свойства транзакции (принцип ACID)
3.	Что такое уровень изоляции Read Uncommitted?
4.	Какие аномалии возможны при уровне изоляции Read Uncommitted?
5.	Как реализуется уровень изоляции Read Uncommitted?
6.	Когда рекомендуется использовать уровень изоляции Read Uncommitted?
7.	Что такое уровень изоляции Read Committed?
8.	Как работает запрос SELECT на уровне изоляции Read Committed?
9.	Могут ли два последовательных оператора SELECT увидеть разные данные даже в рамках одной транзакции?
10.	Для каких случаев не подходит режим Read Committed?

№ п.п.	Содержание вопроса
11.	На каком подходе может основываться реализация чтения зафиксированных данных на уровне изоляции Read Committed?
12	Что такое уровень изоляции Repeatable Read?
13	Как работает уровень изоляции Repeatable Read?
14	Какие аномалии предотвращает уровень изоляции Repeatable Read?
15	Как уровень изоляции Repeatable Read влияет на одновременный конкурентный доступ?
16	Как уровень изоляции Repeatable Read влияет на запросы, которые только читают данные?
17	Что происходит, если приложение использует уровень изоляции Repeatable Read для пишущих транзакций?
18	Что обеспечивает уровень Serializable?
19	Какие аномалии предотвращает уровень Serializable?
20	Как установить уровень изоляции Serializable в СУБД?
21	Какие параметры конфигурации PostgreSQL влияют на производительность?
22	Как выявить медленные запросы в PostgreSQL?
23	Когда стоит использовать индексы для повышения производительности PostgreSQL?
24	Как обновить статистику в PostgreSQL, если данные обновляются часто?

Критерии оценивания опроса:

Диапазон баллов	Описание критерия
85-100	Обучающийся полно излагает материал (отвечает на вопрос), дает правильное определение основных понятий; обнаруживает понимание материала, может обосновать свои суждения, применить знания на практике, привести необходимые примеры не только из учебника, но и самостоятельно составленные; технически правильно излагает материал.
65-84	Обучающийся дает ответ, удовлетворяющий тем же требованиям, что и для оценки «отлично», но допускает 1–2 ошибки, которые сам же исправляет, и 1–2 недочета в последовательности и языковом оформлении излагаемого.
55-64	Обучающийся обнаруживает знание и понимание основных положений данной темы, но излагает материал неполно и допускает неточности в определении понятий или формулировке правил; не умеет достаточно глубоко и доказательно обосновать свои суждения и привести свои

Диапазон баллов	Описание критерия
	примеры; излагает материал непоследовательно и допускает ошибки в языковом оформлении излагаемого.
0-54	Обучающийся обнаруживает незнание вопроса, допускает ошибки в формулировке определений и правил, искажающие их смысл, беспорядочно и неуверенно излагает материал.

КТ – 6.

Тема 11-12

Опрос:

Вопросы для опроса:

№ п.п.	Содержание вопроса
1.	Что такое объектно-ориентированная база данных (ООБД)?
2.	Какие концепции лежат в основе объектно-ориентированного подхода в ООБД: объекты, классы, наследование, атрибуты, сообщения и методы, инкапсуляция?
3.	Какие особенности ООБД позволяют представлять сложные объекты более непосредственным образом, нежели реляционные системы?
4.	Как в ООБД поддерживается средство инверсных связей для выражения взаимных ссылок между двумя объектами (бинарная связь)?
5.	Какие этапы проектирования ООБД выделяют: инфологическое, логическое и физическое проектирование?
6.	Какие задачи решают проектировщики на этапе инфологического проектирования?
7.	Что такое объектно-ориентированная СУБД (ООСУБД)?
8.	Какие функции СУБД с точки зрения пользователя: определение данных, обработка данных, управление данными?
9.	Какие функции СУБД более низкого уровня: управление данными во внешней памяти, управление буферами оперативной памяти, управление транзакциями, ведение журнала изменений в БД?
10.	Как многопользовательские объектно-ориентированные системы поддерживают управление одновременным доступом к объектам?
11.	Какие механизмы защиты данных используются в ООБД?
12.	Как в ООБД обеспечивается целостность данных Как в ООБД обеспечивается целостность данных ?
13.	Подходит ли XML-документ для реализации функций базы данных?
14.	Какие варианты хранения XML-данных в базах данных?
15.	Как XML-документы обрабатываются в базах данных?
16.	Как результаты обработки XML-документа используются в SQL-запросе?

№ п.п.	Содержание вопроса
17	Как XML-данные передаются между разноформатными системами?
18	Как разрабатываются сценарии информационного обмена на основе XML?
19	Как XML-документы защищаются от произвольных или намеренных искажений?
20	Какие меры безопасности применяются при использовании XML для передачи критичных данных, которые должны быть надёжно скрыты от посторонних глаз.

Критерии оценивания опроса:

Диапазон баллов	Описание критерия
85-100	Обучающийся полно излагает материал (отвечает на вопрос), дает правильное определение основных понятий; обнаруживает понимание материала, может обосновать свои суждения, применить знания на практике, привести необходимые примеры не только из учебника, но и самостоятельно составленные; технически правильно излагает материал.
65-84	Обучающийся дает ответ, удовлетворяющий тем же требованиям, что и для оценки «отлично», но допускает 1–2 ошибки, которые сам же исправляет, и 1–2 недочета в последовательности и языковом оформлении излагаемого.
55-64	Обучающийся обнаруживает знание и понимание основных положений данной темы, но излагает материал неполно и допускает неточности в определении понятий или формулировке правил; не умеет достаточно глубоко и доказательно обосновать свои суждения и привести свои примеры; излагает материал непоследовательно и допускает ошибки в языковом оформлении излагаемого.
0-54	Обучающийся обнаруживает незнание вопроса, допускает ошибки в формулировке определений и правил, искажающие их смысл, беспорядочно и неуверенно излагает материал.

5.5. Описание дополнительных материалов и оборудования, необходимых для выполнения проверочных заданий.

Обучающемуся разрешается использование компьютера с установленной СУБД PostgreSQL

6. Формы промежуточной аттестации, критерии и шкала оценивания, типовые оценочные материалы по дисциплине

6.1. Промежуточная аттестация проводится в форме устного зачета с оценкой и экзамена.

6.2. Типовые оценочные материалы промежуточной аттестации.

Тема 1 Введение в базы данных и SQL ОПК-2.2

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Что такое базы данных и зачем они нужны.	
2.	Основные понятия реляционной модели.	
3	Что такое язык SQL.	
4	Концептуальный уровень проектирования.	
5	Логический уровень проектирования.	
6	Физический уровень проектирования.	
7	Проблемы концептуального уровня проектирования.	
8	Проблемы логического уровня проектирования: выбор ключей, восприятие схем данных.	
9	Проблемы физического уровня проектирования: выбор типов данных, масштабируемость, гибкость, обработка логики, скорость работы СУБД.	

1.2. Контрольные задания с ключами правильных ответов.

Задание 1

Привести описание основных сущностей (объектов) ПО. Отбор сущностей производится на основе анализа информационных потребностей. Необходимо привести таблицы описания сущностей (сущностей должно быть не менее 3-х)

Таблица 1. Список сущностей предметной области.

N п.п.	Наименование сущности	Краткое описание

Здесь же приводится отбор атрибутов (не менее 5-ти) для каждого экземпляра сущности. Отбираются только те атрибуты сущностей, которые необходимы для формирования ответов на регламентированные и непредусмотренные запросы. Для каждого объекта следует привести таблицы его атрибутов.

Таблица 2. Список атрибутов.

N п.п.	Наименование атрибута	Краткое описание

На основе анализа информационных запросов следует выявить связи между сущностями. Для выявленных связей также нужно заполнить таблицу 3.

Таблица 3. Список связей ПО.

N п.п.	Наименование связи	Сущности, участвующие в связи	Краткое описание

Задание 2

На основании предоставленных данных описания предметной области построить инфологическую модель базы данных:

- описать классы объектов (сущностей) и их свойства,
- расставить существующие связи между ними,
- на основании табл. 1.3. в письменной форме обосновать типы связей (1:1, 1:M и т.д.).

При графическом построении ИЛМ следует придерживаться единого масштаба для всей схемы. Все прямоугольники, обозначающие классы объектов, должны быть одного размера. Аналогично, все ромбы с именами связей также должны иметь одинаковый размер.

2. Задания комбинированного типа.

2.1. Тестовые задания с обоснованием выбора.

№ п.п.	Содержание задания	Правильный ответ	Аргументы, обосновывающие выбор ответа
1.	Какой элемент базы данных предотвращает		

№ п.п.	Содержание задания	Правильный ответ	Аргументы, обосновывающие выбор ответа
	дублирование и обеспечивает целостность данных Варианты ответов: а) Первичный ключ б) Внешний ключ		
2.	Какие вопросы пытается решить общая теория систем с помощью концептуальных моделей? Варианты ответов: а) Как устроена система?. б) Какие функции она выполняет		

3. Задания закрытого типа.

3.1. Тестовые задания.

Тест 1.

Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитать предложенные варианты ответа.

Выбрать один верный ответ.

Записать только букву выбранного варианта ответа.

1. Совокупность специальным образом организованных данных, хранимых в памяти вычислительной системы и отображающих состояние объектов и их взаимосвязей в рассматриваемой предметной области - это

- а) База данных
- б) СУБД
- с) Банк данных
- д) Информационная система

2. Комплекс языковых и программных средств, предназначенный для создания, ведения и совместного использования БД многими пользователями - это

- а) СУБД
- б) База данных
- с) Словарь данных
- д) Банк данных

3. Реляционная модель представления данных - данные для пользователя передаются в виде
- Таблиц
 - Списков
 - Графа типа дерева
 - Произвольного графа
4. Сетевая модель представления данных - данные представлены с помощью
- Таблиц
 - Списков
 - Упорядоченного графа
 - Произвольного графа
5. Иерархическая модель представления данных - данные представлены в виде
- Таблиц
 - Списков
 - Упорядоченного графа
 - Произвольного графа
6. Атрибут отношения - это
- Строка таблицы
 - Столбец таблицы
 - Таблица
 - Межтабличная связь

Тема 2 Создание рабочей среды ОПК-2.2

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Программа <code>psql</code> — интерактивный терминал PostgreSQL. Установка PostgreSQL в Docker.	
2.	Запуск контейнера PostgreSQL в Docker. Как подключиться к PostgreSQL в контейнере.	

1.2. Контрольные задания с ключами правильных ответов.

Задание 1

Используя репозиторий Ubuntu 20.04 установить PostgreSQL на персональный компьютер с операционной системой Linux.

Задание 2

Выполнить базовую настройку PostgreSQL после установки .

2. Задания комбинированного типа.

2.1. Тестовые задания с обоснованием выбора.

№ п.п.	Содержание задания	Правильный ответ	Аргументы, обосновывающие выбор ответа
1.	При установке PostgreSQL в Docker возникла ошибка ERROR: invalid locale name: "ru_RU.UTF-8" Как ее можно устранить? Варианты ответов: а) необходимо сгенерировать локаль командой: locale-gen.ru_RU.UTF-8 б) Нужно либо использовать кодировку как у шаблона, либо использовать шаблон template0 с добавлением TEMPLATE = template0		
2.	Какие команды нужно выполнить для установки СУБД на Ubuntu/Debian и CentOS/RHEL Варианты ответов: а) sudo apt install -y postgresql-common б) sudo yum install postgresql-server postgresql-contrib		

Тема 3 Основные операции с таблицами ОПК-2.2

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Таблицы в PostgreSQL: типы и операции.	
2	Типы таблиц в PostgreSQL: обычные таблицы, секционированные таблицы, временные таблицы SQL, системные таблицы, широкие таблицы, связанные таблицы SQL.	
3	Основные операции над таблицами в PostgreSQL: создание таблиц, вставка данных, выборка данных, обновление данных, удаление данных.	

1.2. Контрольные задания с ключами правильных ответов.

Задание 1

Предположим, что возникла необходимость хранить в одном столбце таблицы данные, представленные с различной точностью. Это могут быть, например, результаты физических измерений разнородных показателей или различные медицинские показатели здоровья пациентов (результаты анализов). В таком случае можно использовать тип `numeric` без указания масштаба и точности. Команда для создания таблицы может быть, например, такой: `CREATE TABLE test_numeric ( measurement numeric, description text );` Если у вас в базе данных уже есть таблица с таким же именем, то можно предварительно ее удалить с помощью команды `DROP TABLE test_numeric;` Вставьте в таблицу несколько строк: `INSERT INTO test_numeric VALUES ( 1234567890.0987654321, 'Точность 20 знаков, масштаб 10 знаков' );` `INSERT INTO test_numeric VALUES ( 1.5, 'Точность 2 знака, масштаб 1 знак' );` `INSERT INTO test_numeric VALUES ( 0.12345678901234567890, 'Точность 21 знак, масштаб 20 знаков' );` `INSERT INTO test_numeric VALUES ( 1234567890, 'Точность 10 знаков, масштаб 0 знаков (целое число)' );` Теперь сделайте выборку из таблицы и посмотрите, что все эти разнообразные значения сохранены именно в том виде, как вы их вводили.

Задание 2

Можно с высокой степенью уверенности предположить, что при прибавлении интервалов к датам и временным отметкам PostgreSQL учитывает тот факт, что различные месяцы имеют различное число дней. Но как это реализуется на практике? Например, что получится при прибавлении интервала в 1 месяц к

последнему дню января и к последнему дню февраля? Сначала сделайте обобщенные предположения о результатах следующих двух команд, а затем проверьте предположения на практике и проанализируйте полученные результаты:

```
SELECT ( '2026-01-31'::date + '1 mon'::interval ) AS new_date;
SELECT ( '2026-02-29'::date + '1 mon'::interval ) AS new_date;
```

2. Задания закрытого типа.

2.1. Тестовые задания.

Тест 1.

Внимательно прочитайте текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитайте предложенные варианты ответа.

Записью в реляционных базах данных называют:

- a) Ячейку;
- b) Столбец таблицы;
- c) Имя поля;
- d) Строку таблицы.

Поле, значение которого не повторяется в различных записях, называется:

- a) Составным ключом;
- b) Типом поля;
- c) Главным ключом;
- d) Именем поля.

Характеристики типов данных. Убери лишнее.

1	Текстовый;		6	денежный;
2	Поле MEMO		7	словесный;
3	Числовой;		8	дата/время;
4	Функциональный;		9	поле NEMO;
5	Дата/число;		10	счетчик.

Поля реляционной базы данных:

- a) автоматически нумеруются;
- b) именуется пользователем произвольно с определенными ограничениями;
- c) именуется по правилам, специфичным для каждой конкретной СУБД;
- d) нумеруются по правилам, специфичным для каждой конкретной СУБД.

Значение выражения $0,7-3>2$ относится к следующему типу данных:

- a) числовому;
- b) логическому;

- c) *символьному;*
d) *текстовому.*
В чем состоит особенность поля "счетчик"?
a) *служит для ввода числовых данных;*
b) *служит для ввода действительных чисел;*
c) *данные хранятся не в поле, а в другом месте, а в поле хранится только указатель на то, где расположен текст;*
d) *имеет ограниченный размер;*
e) *имеет свойство автоматического наращивания.*
В чем состоит особенность поля "мемо"?
a) *служит для ввода числовых данных;*
b) *служит для ввода действительных чисел;*
c) *данные хранятся не в поле, а в другом месте, а в поле хранится только указатель на то, где расположен текст;*
d) *имеет ограниченный размер;*
e) *имеет свойство автоматического наращивания.*
Какое поле можно считать уникальным?
a) *поле, значения в котором не могут повторяться;*
b) *поле, которое носит уникальное имя;*
c) *поле, значение которого имеют свойство наращивания.*

Тема 4 Типы данных СУБД PostgreSQL ОПК-2.2

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Числовые типы в PostgreSQL.	
2.	Целочисленные типы в PostgreSQL.	
3	Числа с произвольной точностью в PostgreSQL.	
4	Типы с плавающей точкой в PostgreSQL.	
5	Последовательные типы в PostgreSQL.	
6	Денежные типы в PostgreSQL.	
7	Символьные типы в PostgreSQL.	
8	Двоичные типы данных: Шестнадцатеричный	

№ п.п.	Вопрос	Ответ
	формат bytea; Формат спецпоследовательностей bytea.	
9	Типы даты/времени: Ввод даты/времени; Вывод даты/времени; Часовые пояса; Ввод интервалов; Вывод интервалов в PostgreSQL.	
10	Логический тип в PostgreSQL.	
11	Типы перечислений в PostgreSQL. Объявление перечислений: Порядок; Безопасность типа: Тонкости реализации	
12	Геометрические типы в PostgreSQL: Точки; Прямые; Отрезки; Прямоугольники; Пути; Многоугольники; Окружности.	
13	Типы, описывающие сетевые адреса в PostgreSQL: inet; cidr; Различия inet и cidr; macaddr; macaddr8.	
14	Битовые строки в PostgreSQL.	
15	Типы, предназначенные для текстового поиска в PostgreSQL: tsvector; tsquery; Тип UUID.	
16	Тип XML в PostgreSQL: Создание XML-значений. Обработка кодировки. Обращение к XML-значениям.	
17	Типы JSON в PostgreSQL: Синтаксис вводимых и выводимых значений JSON. Проектирование документов JSON. Проверки на вхождение и существование jsonb. Индексация jsonb. Обращение по индексу к элементам jsonb. Трансформации. Тип jsonpath.	

1.2. Контрольные задания с ключами правильных ответов.

Задание 1

Подумайте, какие представления было бы целесообразно создать для нашей базы данных «Авиаперевозки». Необходимо учесть наличие различных групп пользователей, например: пилоты, диспетчеры, пассажиры, кассиры. Создайте представления и проверьте их в работе.

Задание 2

Подумайте, какие еще таблицы было бы целесообразно дополнить столбцами типа json/jsonb. Вспомните, что, например, в таблице «Билеты» (tickets) уже есть столбец такого типа — contact_data. Выполните модификации таблиц и измените в них одну-две строки для проверки правильности ваших решений.

2. Задания закрытого типа.

2.1. Тестовые задания.

Тест 1.

Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитать предложенные варианты ответа.

Записью в реляционных базах данных называют:

- a) Ячейку;*
- b) Столбец таблицы;*
- c) Имя поля;*
- d) Строку таблицы.*

Поле, значение которого не повторяется в различных записях, называется:

- a) Составным ключом;*
- b) Типом поля;*
- c) Главным ключом;*
- d) Именем поля.*

Поля реляционной базы данных:

- a) автоматически нумеруются;*
- b) именуется пользователем произвольно с определенными ограничениями;*
- c) именуется по правилам, специфичным для каждой конкретной СУБД;*
- d) нумеруются по правилам, специфичным для каждой конкретной СУБД.*

Тест 2.

Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитать предложенные варианты ответа.

Записью в реляционных базах данных называют:

Значение выражения $0,7-3>2$ относится к следующему типу данных:

- a) *числовому;*
- b) *логическому;*
- c) *символьному;*
- d) *текстовому.*

В чем состоит особенность поля "счетчик"?

- a) *служит для ввода числовых данных;*
- b) *служит для ввода действительных чисел;*
- c) *данные хранятся не в поле, а в другом месте, а в поле хранится только указатель на то, где расположен текст;*
- d) *имеет ограниченный размер;*
- e) *имеет свойство автоматического наращивания.*

В чем состоит особенность поля "мемо"?

- a) *служит для ввода числовых данных;*
- b) *служит для ввода действительных чисел;*
- c) *данные хранятся не в поле, а в другом месте, а в поле хранится только указатель на то, где расположен текст;*
- d) *имеет ограниченный размер;*
- e) *имеет свойство автоматического наращивания.*

Какое поле можно считать уникальным?

- a) *поле, значения в котором не могут повторяться;*
- b) *поле, которое носит уникальное имя;*
- c) *поле, значение которого имеют свойство наращивания.*

Тема 5 Основы языка определения данных ОПК-2.2

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Какие группы операторов выделяются в составе языка PostgreSQL?	
2.	Дайте неформальное определение основных понятий	

№ п.п.	Вопрос	Ответ
	реляционной модели данных в PostgreSQL?	
3	Для чего нужны внешние ключи в реляционных таблицах?	
4	Что такое потенциальный ключ?	
5	Предложите пример избыточного потенциального ключа в PostgreSQL?	

1.2. Контрольные задания с ключами правильных ответов.

Задание 1

А с помощью какого запроса можно получить результат в таком виде?

```
departure_city | arrival_city | count
-----+-----+-----
Москва | Санкт-Петербург | 12
```

Задание 2

Модифицируйте запрос, добавив в него столбец `level` (можно назвать его и `iteration`). Этот столбец должен содержать номер текущей итерации, поэтому нужно увеличивать его значение на единицу на каждом шаге. Не забудьте задать начальное значение для добавленного столбца в предложении `VALUES`. Задание 2. Для завершения экспериментов замените `UNION ALL` на `UNION` и выполните запрос. Сравните этот результат с предыдущим, когда мы использовали `UNION ALL`.

2. Задания закрытого типа.

2.1. Тестовые задания.

Тест 1.

Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитать предложенные варианты ответа.

ROLLBACK — это структура запроса на уровне?

- a) Язык Управления транзакциями
- b) Язык управления данными
- c) Язык определения данных
- d) Язык манипулирования данными

Сколько таблиц можно объединить с помощью JOIN?

- a) Один
- b) Два
- c) Три
- d) Все вышеперечисленное

Что из перечисленного ниже верно в отношении изменения строк в таблице?

- a) Вы можете изменить несколько столбцов.
- b) Вы можете обновить некоторые строки в таблице на основе значений из другой таблицы.
- c) Если вы попытаетесь обновить запись, связанную с ограничением целостности, возникнет ошибка.
- d) Все вышеперечисленное.

Какая команда используется для резервного копирования базы данных кластера?

- a) `pg_dumpall`
- b) резервная копия `pg_basebackup`
- c) И (A), и (B)
- d) Ничего из вышеперечисленного

Тест 2.

Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитать предложенные варианты ответа.

Как вы называете приложение, которое делает запросы сервера PostgreSQL?

- a) Рабочая станция
- b) Тонкий клиент
- c) Интерфейс
- d) Клиент

Что такое обертка вокруг команды SQL Создать базу данных?

- a) `newdb`
- b) `add_db`
- c) `New_db`
- d) Создан б

Триггеры могут быть настроены для выполнения при выполнении какой из следующих операций:

- a) Все вышеперечисленное

- b) Удалить заявления
- c) Вставьте заявления
- d) Обновлять заявления

Основная команда PSQL для перечисления таблиц есть?

- a) \делать
- b) \час
- c) \ dt
- d) \ dt

Тема 6 Запросы ОПК-2.2

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Как выбрать значения из одного или нескольких столбцов в PostgreSQL?	
2.	Как ограничить результаты выборки с помощью WHERE в PostgreSQL?	
3	Как использовать предложение ORDER BY для сортировки результатов выборки в PostgreSQL?	
4	Как объединить строки из двух или более таблиц на основе связанного столбца между ними в PostgreSQL?	
5	Как создать триггер в PostgreSQL?	
6	Как учитывать типы триггеров в PostgreSQL	

1.2. Контрольные задания с ключами правильных ответов.

Задание1

В каких случаях запрос может вернуть не всё содержимое таблицы?
(parent_id INTEGER, таблица наполнена разнообразными данными)

SELECT * FROM any_table **WHERE** parent_id = parent_id;

А как поведет себя запрос ниже? Какие данные он выведет? *PostgreSQL

SELECT * FROM any_table WHERE parent_id IS NOT DISTINCT FROM parent_id;

Задание2

Какой результат будет у запроса?

-- А)

```
SELECT * FROM (  
  SELECT 1  
  UNION ALL  
  SELECT 1  
  ) x(y)  
UNION  
(  
  SELECT 2  
  UNION ALL  
  SELECT 2  
);
```

Задание 3

Выполнятся ли эти запросы? Какие результаты они вернут?

-- А) **SELECT 3/2;**

-- Б) **SELECT min('Какой-то текст'::TEXT), avg('Какой-то текст'::TEXT);**

-- В)* Почему данный запрос может вернуть FALSE, возможно ли такое поведение СУБД?

SELECT 7.2 = (3.8::FLOAT + 3.4)

-- Г) **SELECT (20/25)*25.0;**

2. Задания закрытого типа.

2.1. Тестовые задания.

Тест 1.

Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитать предложенные варианты ответа.

Дана таблица "table_2" (с единственным столбцом "value"(INTEGER)) состоящая из следующих 5 строк:

VALUE
5
5

NULL
5
5

Какой результат вернет запрос:

SELECT (avg(value)*count(*)) - sum(value) **FROM** table_2;

- a) -4
- b) 0
- c) NULL
- d) 5
- e) Вызовет ошибку, т.к. не указан GROUP BY
- f) Ни один из перечисленных

Дана таблица "table_2" (с единственным столбцом "value"(INTEGER)) состоящая из следующих 5 строк:

value
5
5
NULL
5
5

Какой результат вернет запрос:

SELECT (avg(value)*count(*)) - sum(value) **FROM** table_2;

- a) -4
- b) 0
- c) NULL
- d) 5
- e) Вызовет ошибку, т.к. не указан GROUP BY
- f) Ни один из перечисленных

Тема 7 Изменение данных ОПК-2.2

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Как указать таблицу, в которой будут обновлены данные?	
2.	Как перечислить столбцы и их новые значения?	
3	Как определить условие для идентификации строк, которые будут обновлены (предложение WHERE)?	
4	Как использовать подзапросы в UPDATE для назначения значений на основе данных из других таблиц?	

1.2. Контрольные задания с ключами правильных ответов.

Задание1

Предположим, что для какой-то таблицы создан уникальный индекс по двум столбцам: column1 и column2. В таблице есть строка, у которой значение атрибута column1 равно ABC, а значение атрибута column2 — NULL. Мы решили добавить в таблицу еще одну строку с такими же значениями ключевых атрибутов, т. е. column1 — ABC, а column2 — NULL. Как вы думаете, будет ли операция вставки новой строки успешной или завершится с ошибкой? Объясните ваше решение

2. Задания закрытого типа.

2.1. Тестовые задания.

Тест 1.

Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитать предложенные варианты ответа.

1. Заполните пробелы в запросе «SELECT ___, Country FROM ___ », который возвращает имена заказчиков и страны, где они находятся, из таблицы «Customers».

- a) *, Customers
- b) NULL, Customers
- c) Name, Customers

2. Напишите запрос для выборки данных из таблицы «Customers», где условием является проживание заказчика в городе Москва

- a) *SELECT * FROM Customers WHERE City="Moscow"*
- b) *SELECT City="Moscow" FROM Customers*

c) *SELECT Customers WHERE City="Moscow"*

3. В таблице «Employees» содержатся данные об именах, фамилиях и зарплате сотрудников. Напишите запрос, который изменит значение зарплаты с 2000 на 2500 для сотрудника с ID=7.

a) *SET Salary=2500 FROM Salary=2000 FOR ID=7 FROM Employees*

b) *ALTER TABLE Employees Salary=2500 FOR ID=7*

c) *UPDATE Employees SET Salary=2500 WHERE ID=7*

4. В таблице «Animals» базы данных зоопарка содержится информация обо всех обитающих там животных, в том числе о лисах: red fox, grey fox, little fox. Напишите запрос, возвращающий информацию о возрасте лис.

a) *SELECT %fox age FROM Animals*

b) *SELECT age FROM Animals WHERE Animal LIKE «%fox»*

c) *SELECT age FROM %Fox.Animals*

Тест 2.

Внимательно прочитайте текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитайте предложенные варианты ответа.

1. Что возвращает запрос *SELECT FirstName, LastName, Salary FROM Employees Where Salary<(Select AVG(Salary) FROM Employees) ORDER BY Salary DESC*?

a) *Имена, фамилии и зарплаты сотрудников, значения которых соответствуют среднему значению среди всех сотрудников*

b) *Имена, фамилии сотрудников и их среднюю зарплату за весь период работы, с выполнением сортировки по убыванию*

c) *Имена, фамилии и зарплаты сотрудников, для которых справедливо условие, что их зарплата ниже средней, с выполнением сортировки зарплаты по убыванию*

2. Напишите запрос, возвращающий значения из колонки «FirstName» таблицы «Users».

a) *SELECT FirstName FROM Users*

b) *SELECT FirstName.Users*

c) *SELECT * FROM Users.FirstName*

3. Напишите запрос, возвращающий информацию о заказчиках, проживающих в одном из городов: Москва, Тбилиси, Львов.

- a) *SELECT Moscow, Tbilisi, Lvov FROM Customers*
- b) *SELECT * FROM Customers WHERE City IN ('Moscow', 'Tbilisi', 'Lvov')*
- c) *SELECT City IN ('Moscow', 'Tbilisi', 'Lvov') FROM Customers*

4. Какая команда используется для объединения результатов запроса без удаления дубликатов?

- a) *UNION*
- b) *UNION ALL*
- c) *FULL JOIN*

Тема 8 Индексы

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Зачем нужны индексы в PostgreSQL?	
2.	Как индексы помогают ускорить выполнение запросов на выборку данных (SELECT), особенно на больших таблицах?	
3	Какие типы индексов поддерживает PostgreSQL?	
4	Как создать индекс в PostgreSQL	
5	Как оптимизировать работу индексов в PostgreSQL?	

1.2. Контрольные задания с ключами правильных ответов.

Задание 1

Создайте индекс по столбцу `fare_conditions`. Конечно, в реальной ситуации такой индекс вряд ли целесообразен, но нам он нужен для экспериментов. Прodelайте те же эксперименты с таблицей `ticket_flights`. Будет ли различаться среднее время выполнения запросов для различных значений атрибута `fare_conditions`? Почему это имеет место?

2. Задания закрытого типа.

2.1. Тестовые задания.

Тест 1.

Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитать предложенные варианты ответа.

1. Каково основное назначение индекса в PostgreSQL?
 - a) Для хранения данных
 - b) Для повышения производительности запросов
 - c) Для обеспечения соблюдения ограничений
 - d) Для управления правами пользователя

2. Какой тип индекса используется по умолчанию в PostgreSQL?
 - a) B-tree
 - b) Hash
 - c) GIN
 - d) GICN

3. Какая команда используется для создания индекса в PostgreSQL?
 - a) CREATE-INDEX
 - b) ADD- INDEX
 - c) NEW- INDEX
 - d) INSERT INDEX

4. Какой тип индекса особенно полезен для полнотекстового поиска?
 - a) B-tree
 - b) GIN
 - c) Hash
 - d) BTREE

Тема 9 Транзакция. ОПК-2.2, ОПК ОС-10.2

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Что такое транзакция в PostgreSQL?	
2.	Какие свойства транзакций в PostgreSQL (ACID)?	
3	Что означает согласованность (Consistency)?	
4	Что означает изоляция (Isolation)?	

№ п.п.	Вопрос	Ответ
5	Как управлять транзакциями в PostgreSQL с помощью SQL-команд?	
6	Как обрабатывать взаимоблокировки в PostgreSQL?	
7	Как минимизировать взаимоблокировки?	

1.2. Контрольные задания с ключами правильных ответов.

Задание 1

Измените сценарий выполнения транзакций: в первой транзакции вместо фиксации изменений выполните их отмену с помощью команды ROLLBACK и посмотрите, будет ли удалена строка и какая именно.

DELETE

FROM aircrafts_tmp

WHERE range < 2000;

*SELECT **

FROM aircrafts_tmp;

Задание 2

Когда говорят о таком феномене, как потерянное обновление, то зачастую в качестве примера приводится операция UPDATE, в которой значение какого-то атрибута изменяется с применением одного из действий арифметики. Например:

UPDATE aircrafts_tmp

SET range = range + 200

WHERE aircraft_code = 'CR2';

При выполнении двух и более подобных обновлений в рамках параллельных транзакций, использующих, например, уровень изоляции Read Committed, будут учтены все такие изменения (что и было показано в тексте главы). Очевидно, что потерянного обновления не происходит. Предположим, что в одной транзакции будет просто присваиваться новое значение.

Очевидно, что сохранится только одно из значений атрибута range. Можно ли говорить, что в такой ситуации имеет место потерянное обновление? Если оно имеет место, то что можно предпринять для его недопущения? Обоснуйте ваш ответ.

2. Задания закрытого типа.

2.1. Тестовые задания.

Тест 1.

Внимательно прочитать текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитать предложенные варианты ответа.

1. Что такое транзакция в PostgreSQL?

- A. Одна атомарная операция
- B. Последовательность операций, выполняемых одна за другой
- C. Процесс резервного копирования базы данных
- D. Пользовательская функция

2. Какая команда используется для запуска транзакции в PostgreSQL?

- A. BEGIN
- B. START
- C. INIT
- D. OPEN

3. Что делает команда COMMIT?

- A. Завершает текущую транзакцию и сохраняет изменения
- B. Отменяет все изменения, внесённые во время транзакции
- C. Начинает новую транзакцию
- D. Отображает статус транзакции

4. Что происходит при выполнении команды ROLLBACK?

- A. Изменения сохранены
- B. Изменения отменены
- C. Транзакция приостановлена
- D. Начинается новая транзакция

5. Какой уровень изоляции используется по умолчанию в PostgreSQL?

- A. Прочитать незафиксированное
- B. Прочитано с согласия
- C. Повторяющееся чтение
- D. Сериализуемый

Тема 10 Повышение производительности. ОПК-2.2, ОПК ОС-10.2, ОПК ОС-11.1

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Какие ключевые параметры конфигурации PostgreSQL нужно настроить для	

№ п.п.	Вопрос	Ответ
	повышения производительности?	
2.	Как выявить медленные запросы в PostgreSQL?	
3	Как оптимизировать структуру запросов	
4	Как использовать представления и материализованные представления для кэширования и повторного использования сложных запросов.	
5	Как отслеживать статистику выполнения запросов с помощью расширения pg_stat_statements?	

1.2. Контрольные задания с ключами правильных ответов.

Задание 1

Самостоятельно выполните команду *EXPLAIN* для запроса, содержащего общее табличное выражение (CTE). Посмотрите, на каком уровне находится узел плана, отвечающий за это выражение, и как он оформлен. Учтите, что общие табличные выражения всегда материализуются, то есть вычисляются однократно, а результат их вычисления сохраняется в памяти, после чего все последующие обращения в рамках запроса направляются уже к этому материализованному результату.

ОБЪЯСНЕНИЕ

```
WITH city_from AS (
  ВЫБРАТЬ ОТДЕЛЬНО город
  ИЗ аэропортов
)
ВЫБРАТЬ количество(*)
ИЗ city_from AS a1
ПРИСОЕДИНИТЬ city_from AS a2
ПО a1.город <> a2.город;
```

Задание 2

Выполните команду *EXPLAIN* для запроса, в котором используется одна из оконных функций. Найдите в плане выполнения запроса узел с именем *WindowAgg*. Попробуйте объяснить, почему он находится именно на этом уровне в плане.

Задание 3

Замена коррелированного подзапроса соединением таблиц является одним из способов повышения производительности. Предположим, что мы задались вопросом: сколько маршрутов обслуживают самолеты каждого типа? При этом нужно учитывать, что может иметь место такая ситуация, когда самолеты какого-либо типа не обслуживают ни одного маршрута. Поэтому необходимо использовать не только представление «Маршруты» (routes), но и таблицу «Самолеты» (aircrafts). Это первый вариант запроса, в нем используется коррелированный подзапрос.

Тема 11 Объектно-ориентированные базы данных ОПК-2.2, ОПК ОС-10.2, ОПК ОС-11.1

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Что такое объектно-ориентированные базы данных?	
2.	На каких концепциях базируется объектно-ориентированный подход?	
3	Какие обязательные характеристики есть у объектно-ориентированных баз данных?	
4	В чем основное отличие объектно-ориентированных баз от реляционных	
5	В каких случаях рекомендованы объектно-ориентированные базы данных?	
6	Какие преимущества есть у таких баз данных перед реляционными системами?	
7	Какие возможности предоставляют объектно-ориентированные базы данных пользователю?	
8	Почему в объектно-ориентированных базах данных нет потребности в	

№ п.п.	Вопрос	Ответ
	определяемых пользователями ключах?	
9	Какие типы сравнения разработаны в модели объектно-ориентированных баз данных?	
10	Как происходит отображение классов на таблицы и реализация наследования в объектно-ориентированных базах данных.	

1.2. Контрольные задания с ключами правильных ответов.

Учебную базу данных можно наполнить информацией, функциями и триггерами с помощью команд, выполняемых в командной строке Debian:

```
createdb ais -U postgres  
psql -d ais -f adj_list.sql -U postgres
```

Задание 1

Выполните проверку структуры дерева на предмет отсутствия циклов с помощью функции tree_test().

Задание 2

Выполните обход дерева организационной структуры снизу вверх, начиная с конкретного узла, можно с помощью функции up_tree_traversal() либо функции up_tree_traversal2().

Задание 3

Выполните операцию удаления поддерева с помощью функции delete_subtree(). Параметром функции является код работника. Аналогично работе с функцией up_tree_traversal() используйте подзапрос для получения кода работника по его имени. После удаления поддерева посмотрите, что стало с организационной структурой, с помощью двух представлений Personnel_org_chart и Create_paths.

Задание 4

Если в таблице «Организационная структура» осталось мало данных, то дополните ее данными и выполните удаление элемента иерархии и продвижение дочерних элементов на один уровень вверх (т. е. к «бабушке»).

Аналогично работе с функцией up_tree_traversal() используйте подзапрос для получения кода работника по его имени. После удаления элемента иерархии посмотрите, что стало с организационной структурой, с помощью двух представлений Personnel_org_chart и Create_paths.

Задание 5

Воспользуйтесь технической документацией на PostgreSQL, глава «PL/pgSQL – SQL Procedural Language». Напишите небольшую функцию с применением курсора.

Тема 12 Использование XML при работе с БД. ОПК-2.2, ОПК ОС-11.1

1. Задания открытого типа.

1.1. Вопросы открытого типа.

№ п.п.	Вопрос	Ответ
1.	Что такое XML-ориентированная база данных?	
2.	Как связан язык XML с реляционным?	
3	Почему на основе XML может и должна быть построена модель данных?	
4	Что такое XML-документ и из чего он состоит?	
5	Почему при использовании XML для хранения данных важно заботиться о масштабируемости и безопасности?	
6	Как многие реляционные базы данных позволяют пользователям получать XML из данных, хранящихся в реляционных таблицах?	
7	Как многие СУРБД получают XML-данные от пользователей, преобразуют их в реляционный вид и сохраняют в реляционных таблицах?	

2. Задания закрытого типа.

2.1. Тестовые задания.

Тест 1.

Внимательно прочитайте текст задания и понять, что в качестве ответа ожидается только один из предложенных вариантов.

Внимательно прочитать предложенные варианты ответа.

1. Для чего в XML используется указание DTD?
 - a) *Для указания информации об авторе данного XML-документа*
 - b) *Для того, чтобы ограничить структуру XML-документа*
 - c) *Используется для задания заголовка XML-документов*
 - d) *Для указания контактной информации адресата данного XML-документа*
 - e) *Для того, чтобы расширить структуру XML-документа*
2. Сущности в XML - это...
 - a) *символы, которые имеют особые значения и являются служебными*
 - b) *объекты XML*
 - c) *атрибуты XML*
 - d) *элементы XML*
3. Что означает аббревиатура XML?
 - a) *Расширяемый язык разметки*
 - b) *Дополнительный язык разметки*
 - c) *Исполняемый язык разметки*
 - d) *Расширенный язык разметки*
4. Что из перечисленного ниже является ключевым преимуществом использования XML-баз данных?
 - a) *Более высокая производительность по сравнению с реляционными базами данных*
 - b) *Поддержка сложных типов данных*
 - c) *Ограниченное хранилище данных*
 - d) *Нет поддержки запросов*
5. Какой тип структуры данных в основном используется в XML-базах данных?
 - a) *Реляционные таблицы*
 - b) *Иерархическая структура*
 - c) *Плоские файлы*
 - d) *Пары «ключ-значение»*
6. Что из перечисленного НЕ является характеристикой XML-баз данных?
 - a) *Гибкость*
 - b) *Дизайн без схем*
 - c) *Строгая целостность данных*
 - d) *Самоописательный характер*
7. В каких сферах обычно используются XML-базы данных?
 - a) *Веб-сервисы*
 - b) *Игры*
 - c) *Приложения для работы с электронными таблицами*
 - d) *Настольное программное обеспечение*

6.3. Критерии и шкала оценивания на основе БРС.

Критерии и балльная шкала определяются преподавателем

КРИТЕРИИ ОЦЕНИВАНИЯ	РЕЗУЛЬТАТ В БАЛЛАХ
<i>Дан полный, в логической последовательности развернутый ответ на поставленный вопрос, где он продемонстрировал знания предмета в полном объеме учебной программы, достаточно глубоко осмысливает дисциплину, самостоятельно, и исчерпывающе отвечает на дополнительные вопросы, приводит собственные примеры по проблематике поставленного вопроса, решил предложенные практические задания без ошибок</i>	40
<i>Дан развернутый ответ на поставленный вопрос, где студент демонстрирует знания, приобретенные на лекционных и семинарских занятиях, а также полученные посредством изучения обязательных учебных материалов по курсу, дает аргументированные ответы, приводит примеры, в ответе присутствует свободное владение монологической речью, логичность и последовательность ответа. Однако допускается неточность в ответе. Решил предложенные практические задания с небольшими неточностями.</i>	30-39
<i>Дан ответ, свидетельствующий в основном о знании процессов изучаемой дисциплины, отличающийся недостаточной глубиной и полнотой раскрытия темы, знанием основных вопросов теории, слабо сформированными навыками анализа явлений, процессов, недостаточным умением давать аргументированные ответы и приводить примеры, недостаточно свободным владением монологической речью, логичностью и последовательностью ответа. Допускается несколько ошибок в содержании ответа и решении практических заданий.</i>	20-29
<i>Дан ответ, который содержит ряд серьезных неточностей, обнаруживающий незнание процессов изучаемой предметной области, отличающийся неглубоким раскрытием темы, незнанием основных вопросов теории, несформированными навыками анализа явлений, процессов, неумением давать аргументированные ответы, слабым владением монологической речью, отсутствием логичности и последовательности. Выводы поверхностны. Решение практических заданий не выполнено, т.е. студент не способен ответить на вопросы даже при дополнительных наводящих вопросах преподавателя.</i>	0-19

6.4. Описание дополнительных материалов и оборудования, необходимых для выполнения проверочных заданий.

Обучающемуся разрешается использование компьютера с

7. Методические материалы по освоению дисциплины (модуля)

Обучение по дисциплине «Вычислительные системы, сети и телекоммуникации» предполагает изучение курса на аудиторных занятиях (лекции, практические) и самостоятельной работы студентов. Практические занятия дисциплины «Базы данных» предполагают их проведение в различных формах с целью выявления полученных знаний, умений, навыков и компетенций.

Подготовка к лекции студентами заключается в следующем:

- повторить материал предыдущей лекции, прочитав его повторно;
- ознакомиться с темой предстоящей лекции (в рабочей программе учебной дисциплины);
- ознакомиться с учебными материалами по данной теме в соответствии с предложенным списком литературы в рабочей программе учебной дисциплины или с электронными материалами, предложенными лектором;
- записать возможные вопросы, которые можно будет задать лектору.

Подготовка к практическим (семинарским) занятиям:

- внимательно прочитать материал лекций, относящихся к данному занятию, ознакомиться с учебными материалами, включая электронные в соответствии с предложенным списком литературы в рабочей программе учебной дисциплины;
- подготовить развернутые ответы на вопросы, предложенные в рабочей программе дисциплины для обсуждения;
- выполнить задания, если они предусмотрены в письменной форме;
- понять, что для вас осталось неясными и постараться получить на них ответ заранее;
- готовиться к практическим/семинарским занятиям можно как индивидуально, так и в составе малой группы;
- рабочую программу учебной дисциплины необходимо использовать в качестве основного ориентира в организации обучения;
- электронная версия рабочей программы по дисциплине размещена на сайте Нижегородского института управления, к ней предоставлен авторизованный доступ.

Самостоятельная работа обучающегося:

Самостоятельная работа призвана закрепить теоретические знания и практические навыки, полученные на лекциях, практических и семинарских занятиях.

Самостоятельная работа проводится с целью:

- систематизации и закрепления полученных теоретических знаний и практических умений обучающихся;
- углубления и расширения теоретических знаний;
- формирования умений использовать справочную документацию и специальную литературу;
- развития познавательных способностей и активности: творческой инициативы, самостоятельности, ответственности и организованности;
- формирования самостоятельности мышления, способностей к саморазвитию, самосовершенствованию и самореализации;
- развития исследовательских умений.

Эффективность лекционных, семинарских и практических занятий по дисциплине во многом зависит от качества самостоятельной работы обучающихся, от их самоподготовки.

Часть времени, отведенного на самостоятельную работу, должна использоваться на подготовку к аудиторным занятиям, другая часть на выполнение домашней работы, осмысление и оформление результатов практических занятий.

При подготовке к занятиям студенту полезно:

- изучить теоретический материал по данной теме (конспект занятия);
- ознакомиться с литературой, рекомендованной преподавателем;
- выполнить задания, предложенные преподавателем, к занятию;
- составить перечень вопросов, вызывающих затруднения, неясности или сомнения, обсудить их с преподавателем или на занятии;
- заниматься самостоятельным поиском дополнительной литературы по изучаемой теме.

Подготовка к зачету/ экзамену. К зачету/ экзамену необходимо готовиться целенаправленно, регулярно, систематически и с первых дней обучения по данной дисциплине. В самом начале учебного курса познакомьтесь со следующей учебно-методической документацией:

- программой дисциплины;
- перечнем знаний и умений, которыми студент должен владеть;
- тематическими планами лекций, семинарских занятий;
- учебником, учебными пособиями по дисциплине, а также электронными ресурсами;
- перечнем и тематикой письменных работ, а также методическими рекомендациями по их выполнению;
- перечнем вопросов.

Систематическое выполнение всех видов заданий на лекциях, практических занятиях, а также самостоятельная работа позволит успешно освоить дисциплину и создать хорошую базу для сдачи зачета/ экзамена.

Вопросы для самостоятельной подготовки к занятиям

1. Опишите порядок установки СУБД PostgreSQL для операционной системы Windows.
2. Что необходимо указать пользователю при первом запуске SQL Shell (psql)?
3. Какая команда используется для создания новых пользователей в PostgreSQL?
4. С помощью какой команды выполняется модификация существующих пользователей?
5. Какую команду в PostgreSQL можно использовать для создания баз данных?
6. С помощью какой команды можно удалить базу данных?
7. Какая команда используется для изменения атрибутов базы данных?
8. Какая команда используется для создания таблиц?
9. С помощью какой команды можно получить описание таблицы с указанием списка ее полей?
10. Каким образом можно вставлять новые строки в таблицу?
11. Какая команда используется для модификации таблиц и полей?
12. Каким образом можно выполнить обновление данных в таблице?
13. Какая команда осуществляет удаление записей из таблиц?
14. Каким образом выполняется объединение таблиц с использованием ключевого слова `inherits`?
15. Что такое ограничение? Как можно задать ограничение?
16. Какие типы ограничений полей существуют?
17. Каким образом можно добавить ограничения в существующую таблицу?
18. С помощью какой команды можно увидеть все ограничения, наложенные на поля таблицы?
19. Как можно удалить ограничение?
20. С какой целью в запросах используется ключевое слово `DISTINCT`?
21. В каких случаях используется представление?
22. С помощью какой команды можно создать представление?
23. Перечислите преимущества использования синонимов.

8. Учебная литература и ресурсы информационно-телекоммуникационной сети Интернет

8.1. Основная литература

1. Ивина, Н.Л. Теория и практика проектирования баз данных : учеб.-метод. пособие / Н.Л. Ивина, А.Е. Матвеев; Нижегород. ин-т упр. - Н.Новгород : НИУ РАНХиГС, 2017. - 184 с. - ISBN 978-5-00036-166-5 : 2014-13; 2014-12.
2. Нестеров, С. А. Базы данных : учебник и практикум для вузов / С. А. Нестеров. — 2-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2024. — 258 с. — (Высшее образование). — ISBN 978-5-534-18107-4. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: <https://urait.ru/bcode/536687> (дата обращения: 17.04.2025). - Режим доступа: по подписке.
3. Маркин, А. В. Программирование на SQL : учебник и практикум для вузов / А. В. Маркин. — 3-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2025. — 805 с. — (Высшее образование). — ISBN 978-5-534-18371-9. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: <https://urait.ru/bcode/568900> (дата обращения: 17.04.2025). - Режим доступа: по подписке.
4. Рогов, Е.В. PostgreSQL 16 изнутри : практическое руководство / Е. В. Рогов. – Москва : ДМК Пресс, 2024. - 666 с. – ISBN 978-5-93700-305-8. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2205083> (дата обращения: 17.04.2025). – Режим доступа: по подписке.

8.2. Дополнительная литература

1. Карпова, Т. С. Базы данных: модели, разработка, реализация : учебное пособие / Т. С. Карпова. - Москва : ИНТУИТ, 2016. - 288 с. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2138291> (дата обращения: 17.04.2025). – Режим доступа: по подписке.
2. Стружкин, Н. П. Базы данных: проектирование. Практикум : учебник для вузов / Н. П. Стружкин, В. В. Годин. — Москва : Издательство Юрайт, 2025. — 291 с. — (Высшее образование). — ISBN 978-5-534-00739-8. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: <https://urait.ru/bcode/561215> (дата обращения: 17.04.2025). – Режим доступа: по подписке.
3. Гордеев, С. И. Организация баз данных в 2 ч. Часть 1 : учебник для вузов / С. И. Гордеев, В. Н. Волошина. — 2-е изд., испр. и доп. — Москва : Издательство Юрайт, 2024. — 310 с. — (Высшее образование). — ISBN 978-5-534-04469-0. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: <https://urait.ru/bcode/538593> (дата обращения: 13.04.2025). – Режим доступа: по подписке.

4. Гордеев, С. И. Организация баз данных в 2 ч. Часть 2 : учебник для вузов / С. И. Гордеев, В. Н. Волошина. — 2-е изд., испр. и доп. — Москва : Издательство Юрайт, 2024. — 513 с. — (Высшее образование). — ISBN 978-5-534-04470-6. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: <https://urait.ru/bcode/539672> (дата обращения: 13.04.2025). — Режим доступа: по подписке.

5. Туманов, В. Е. Основы проектирования реляционных баз данных : учебное пособие / В. Е. Туманов. — 4-е изд. — Москва : Интернет-Университет Информационных Технологий (ИНТУИТ), Ай Пи Ар Медиа, 2024. — 502 с. — ISBN 978-5-4497-3329-0. — Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. — URL: <https://www.iprbookshop.ru/142291.html> (дата обращения: 17.04.2025). — Режим доступа: для авторизир. пользователей.

6. Разработка приложений на С# с использованием СУБД PostgreSQL / Васюткина И.А., Трошина Г.В., Бычков М.И. - Новосибирск : НГТУ, 2015. - 143 с.: ISBN 978-5-7782-2699-9. - Текст : электронный. - URL: <https://znanium.com/catalog/product/556925> (дата обращения: 17.04.2025). — Режим доступа: по подписке.

8.3. Нормативные правовые документы и иная правовая информация

1. Федеральный закон «Об образовании в Российской Федерации» от 29.12.2012 №273-ФЗ. // Справочная правовая система КонсультантПлюс: [сайт]. URL: https://www.consultant.ru/document/cons_doc_LAW_140174/ (дата обращения: 14.04.2025).

2. Приказ Министерства науки и высшего образования Российской Федерации от 06.04.2021 №245 «Об утверждении Порядка организации и осуществления образовательной деятельности по образовательным программам высшего образования - программам бакалавриата, программам специалитета, программам магистратуры» // Справочная правовая система КонсультантПлюс: [сайт]. URL: https://www.consultant.ru/document/cons_doc_LAW_393023/ (дата обращения: 14.04.2025).

3. Приказ РАНХиГС Об утверждении положения о единой балльно-рейтинговой системе оценивания успеваемости студентов Академии и ее использовании при проведении текущей и промежуточной аттестации от 12.12.2024 №02-2531.

8.4 Интернет-ресурсы

1. Инновации SQL Server 2019. Использование технологий больших данных и машинного обучения / пер. с англ. Желновой Н. Б. – М.: ДМК Пресс, 2020. – 408 с: https://www.galaktika-dmk.com/upload/iblock/6c2/978_5_97060_595_0.pdf

2. Карпова Т.С. Базы данных: модели, разработка, реализация. Курс интернет-университета информационных технологий (INTUIT.RU). Web: https://www.studmed.ru/view/karpova-ts-bazy-dannyh-modeli-razrabotka-realizaciya_709b37ed11c.html
3. Кузнецов С.Д. Введение в модель данных SQL. Курс интернет-университета информационных технологий (INTUIT.RU). Web: https://www.studmed.ru/kuznecov-s-d-vvedenie-v-model-dannyh-sql_092b572d00f.html
4. Кузнецов С.Д. Введение в реляционные базы данных. Курс интернет-университета информационных технологий (INTUIT.RU). Web: https://www.studmed.ru/kuznecov-s-d-vvedenie-v-model-dannyh-sql_092b572d00f.html
5. PostgreSQL: Документация. Web: <https://postgrespro.ru/docs/postgresql/17/intro-what-is?ysclid=m9l2jabovx906435405>
6. Основы PostgreSQL для начинающих: от установки до первых запросов. Web: <https://tproger.ru/articles/osnovy-postgresql-dlya-nachinayushhih--ot-ustanovki-do-pervyh-zaprosov-250851?ysclid=m9l2la3oq0983231878>
7. Полякова Л.Н. Основы SQL. Курс интернет-университета информационных технологий (INTUIT.RU). Web: https://www.studmed.ru/view/polyakova-lp-osnovy-sql-kurs-lekciy_6124857b59f.html
8. PostgreSQL: Полное руководство для начинающих. Web: https://sky.pro/media/postgresql-polnoe-rukovodstvo-dlya-nachinayushhih/?ysclid=m9l2nusbcs208942492_
9. Облачные базы данных. Web: https://linx.ru/cloud/cloud-database?utm_source=yandex&utm_medium=cpc&utm_campaign=database&type=search&source=none&block=other&position=2&utm_term=cyбд%20postgresql&yclid=13781503866488225791
10. Хостинг с поддержкой PostgreSQL. Web: https://sweb.ru/hosting/cheap/?utm_source=yandex&utm_medium=cpc&utm_campaign=hosting_cpa_cheap&utm_term=SKIDKA30&utm_content=5531303185%7C16773978881&yclid=7830627299113304063

9. Материально-техническая база, информационные технологии, программное обеспечение и информационные справочные системы

9.1. Материально-техническая база

Перечень материально-технического обеспечения:

1. Учебные аудитории, оборудованные для проведения учебных занятий лекционного и семинарского типа, групповых и индивидуальных консультаций, коллоквиумов, мероприятий текущего контроля и промежуточной аттестации, в том числе мультимедийным оборудованием

для демонстрации электронных презентаций и аудио- и видеоматериалов.

2. Компьютерные классы для выполнения групповых тестовых и иных заданий, а также для самостоятельной работы обучающихся оснащенные компьютерной техникой и обеспечением доступа к сети «Интернет» и доступа в электронную информационно-образовательную среду организации.

3. Специализированные аудитории и лаборатории.

4. Библиотека с обеспечением печатными изданиями или электронно-библиотечная система обеспечивающая доступ к электронным изданиям (электронная библиотека).

5. Читальный зал.

6. Технические средства обучения: персональные компьютеры; компьютерные проекторы; звуковые динамики; программные средства, обеспечивающие просмотр видеофайлов в форматах AVI, MPEG-4, DivX, RMVB, WMV и др.

9.2. Информационные технологии, программное обеспечение:

При осуществлении образовательного процесса по дисциплине используется информационные технологии и программное обеспечение:

1. Современная операционная система.

2. Kaspersky Endpoint Security (или аналог).

3. Средство просмотра файлов формата pdf.

4. Современные офисные средства (текстовые и табличные редакторы, средства работы с презентационными материалами и т.д.).

5. Архиватор 7-Zip.

6. Система дистанционного обучения.

7. Автоматизированная библиотечная система.

9.3. Информационные справочные системы:

1. <https://www.urait.ru> – Электронно-библиотечная система [ЭБС] Юрайт;

2. <https://www.iprbookshop.ru> – Электронно-библиотечная система [ЭБС] «IPRSMART» (ранее – IPRBooks)

3. <https://e.lanbook.com> - Электронно-библиотечная система [ЭБС] «Лань».

4. <https://znanium.ru> - Электронно-библиотечная система [ЭБС] «Znanium.com».

5. <https://www.book.ru> - Электронно-библиотечная система [ЭБС] «Book.ru».

6. <https://ibooks.ru> - Электронно-библиотечная система [ЭБС] «ibooks.ru».

7. <https://ranepalib.miflib.ru> - Электронная библиотека издательства «МИФ».

8. <https://eivis.ru> – Ивис (EastView). Полные тексты российских научных и практических журналов, а также газет центральной прессы

России. Доступ с ip-адресов локальной сети Института.

9. <https://grebennikov.ru> - Полные тексты 38 научно-практических журналов по маркетингу, менеджменту, финансам и управлению персоналом ИД «Гребенников». Доступ с IP-адресов локальной сети Института.

10. <http://www.consultant.ru/> - Справочно-правовая система «Консультант».

11. <http://www.garant.ru/> Справочно-правовая система «Гарант».